Politechnika Świętokrzyska

Aplikacje Internetowe
Podstawy Technologii Internetowych

Dr inż. **Wikarek Jarosław**
Katedra Informatyki Stosowanej
e-mail: j.wikarek@tu.kielce.pl
pokój: d408

Wydział Elektrotechniki, Automatyki i Informatyki

XML

XML (skrót od eXtensible Markup Language - rozszerzalny język znaczników) to otwarty standard opracowany przez W3C.

XML

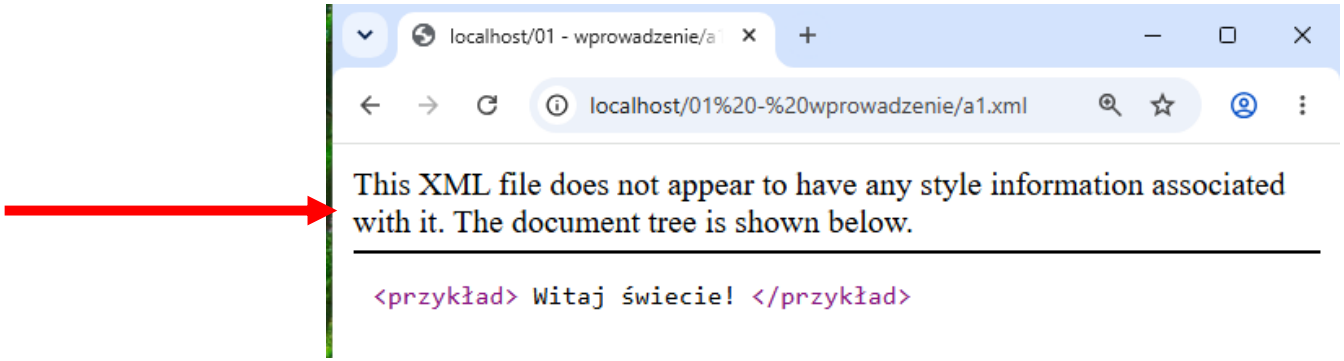
Wprowadzenie

- XML nie jest on kolejnym językiem do przechowywania konkretnych danych, jak np. język HTML (opisujący wygląd stron sieciowych).
- XML to język opisujący dane, czyli metajęzyk.
- W uproszczeniu można powiedzieć, że XML służy do tworzenia innych języków (aplikacji XML) służących do przechowywania informacji.
- XML oddziela dane od sposobu ich publikowania.
- W XML-u dane przechowywane są w sposób tekstowy, opisywane poprzez znaczniki, w obrębie, których te dane się znajdują.
- Znacznik rozpoczyna lewy nawias kątowy <, a kończy prawy >.

Wprowadzenie

- W uproszczeniu można przyjąć, że zestaw znaczników i ich zawartości tworzą dokument XML, będący najczęściej, ale niekoniecznie, plikiem.
- Przykład dokumentu

```
<?xml version="1.0" standalone="yes"?>
<przykład>
    Witaj świecie!
</przykład>
```



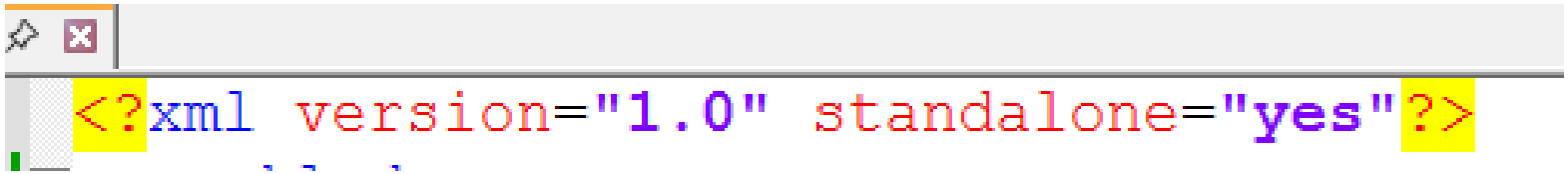
Przykład dokumentu

- Na początku znajduje się prolog dokumentu. W prologu umieszcza się zwykle deklaracje. Może to być standardowa deklaracja XML (pierwsza linia na listingu), czy np. deklaracja arkusza stylów.
- W linii drugiej rozpoczyna się element główny, a wewnątrz niego znajduje się treść „Witaj świecie!” – jego zawartość tekstowa.

```
<?xml version="1.0" standalone="yes"?>
<przykład>
    Witaj świecie!
</przykład>
```

Budowa dokumentu

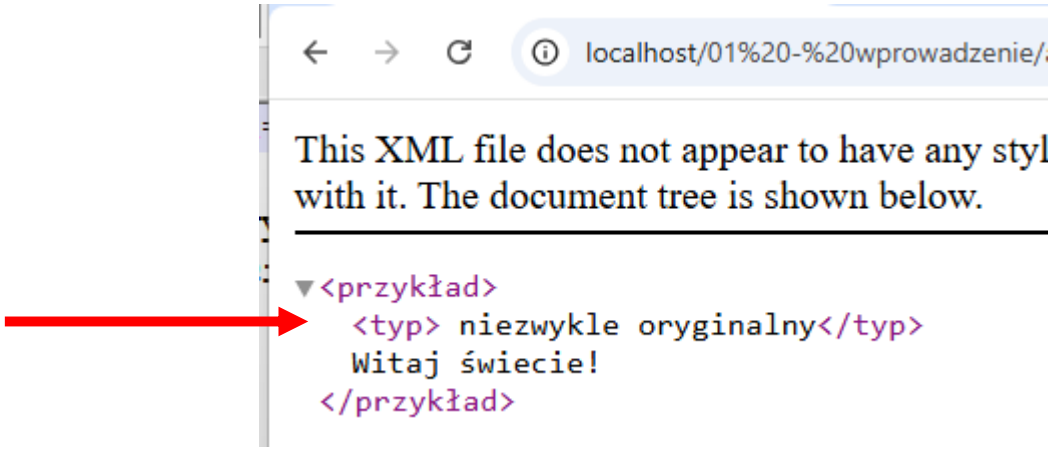
- Prolog nie jest obowiązkową częścią dokumentu.
- Składa się z rozmaitych deklaracji, (które, najczęściej są jednocześnie instrukcjami przetwarzania). Taką deklaracją jest np. standardowa deklaracja XML
  - ✓ **`version`** Jedyny wymagany atrybut — wersja.
  - ✓ **`encoding`** Kodowanie znaków.
  - ✓ **`Standalone`** Dokument samodzielny czy zawiera zewnętrzne źródła



Dodatkowe dane w dokumencie

- Spróbujmy poszerzyć dokument o dalsze informacje – właściwość o nazwie „typ”
  - ✓ Pod element

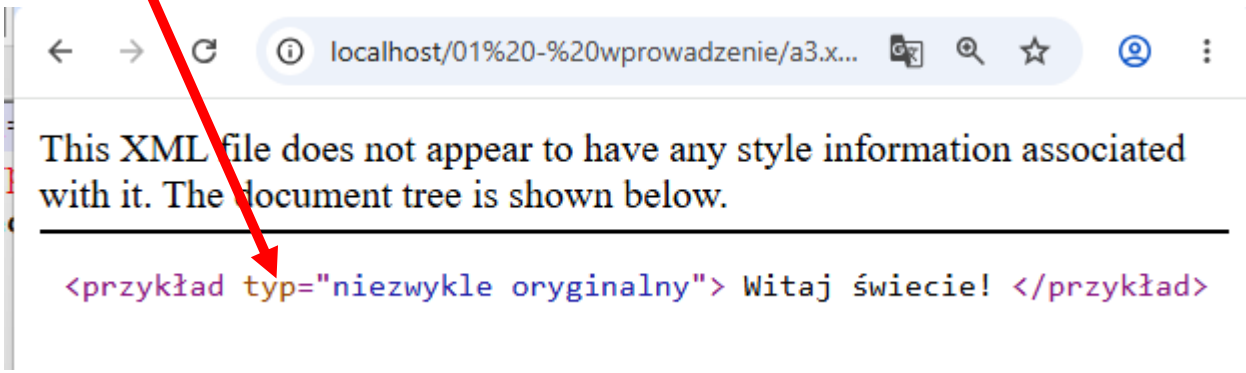
```
<?xml version="1.0" standalone="yes"?>
<przykład>
  <typ> niezwykle oryginalny</typ>
  Witaj świecie!
</przykład>
```



Dodatkowe dane w dokumencie

- Spróbujmy poszerzyć dokument o dalsze informacje – właściwość o nazwie „typ”
  - ✓ Atrybut

```
<?xml version="1.0" standalone="yes"?>
<przykład typ="niezwykle oryginalny">
  Witaj świecie!
</przykład>
```



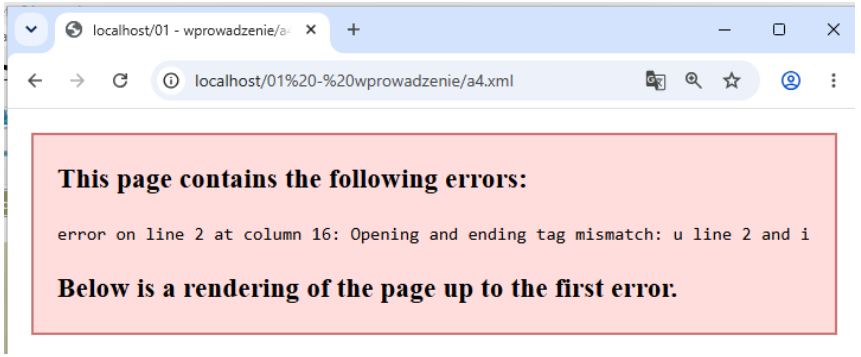
Budowa dokumentu

- Wartość tekstowa nie może bezpośrednio zawierać:
  - ✓ Nawiasów kątowych **< >**
  - ✓ **&** (znak zastrzeżony).
- Wartość atrybutu nie może bezpośrednio zawierać:
  - ✓ Nawiasów kątowych (w atrybucie można jednak użyć nawiasu zamykającego) **<**
  - ✓ Cudzysłowów. **"**
  - ✓ **&** (znak zastrzeżony).

Budowa dokumentu

- Każdy element musi być zamknięty. Należy pamiętać o zachowaniu struktury dokumentu.
  - ✓ Błędne jest:

```
<i><u>dana</i></u>
```

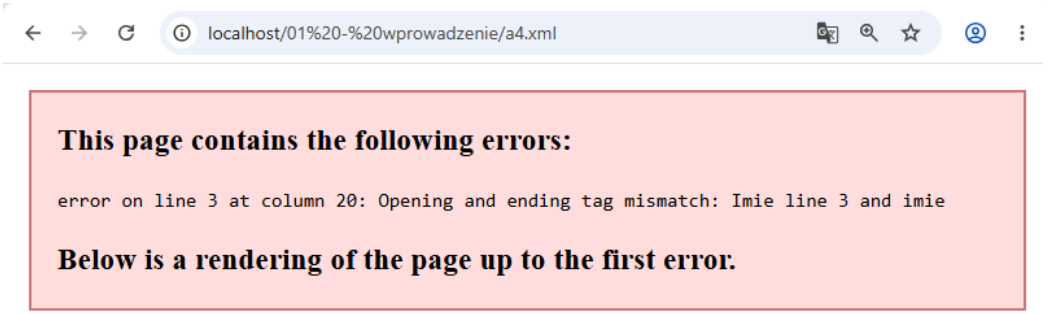


  - ✓ Poprawne jest:

```
<i><u>dana</u></i>
```

- W XML-u rozróżniana jest wielkość liter. Nieprawidłowy będzie, więc kod:

```
<Imie>Ala</imie>
```



Budowa dokumentu

- **Sekcja CDATA.** Pozwalają włączyć do dokumentu tekst, w który ma być zignorowany przez parser.

```
<![CDATA[ Hej! to & jest <właśnie> sekcja
    cdata.  ]]>
```

- **Instrukcje przetwarzania** Są pokrewne sekcjom CDATA, ale lepiej nadają się do „przemycania” w dokumentach XML np. skryptów.

```
<?javascript alert(”To jest <tylko>
   przykład.”); ?>
<?php function printsum($a, $b)
  {echo $a + $b; }
?>
```

Budowa dokumentu

- **Komentarze** Pozwalają na dodanie uwag widocznych (zwykle) tylko w czasie edycji dokumentu.

```
<!-- <bez> komentarza --!>
```

- **Encje** Są czymś w rodzaju szablonów tekstu.
  - ✓ `<` oznacza znak <.
  - ✓ `>` oznacza znak >.
  - ✓ `'` oznacza znak ’ (apostrof).
  - ✓ `"` oznacza znak ” (cudzysłów podwójny).
  - ✓ `&` oznacza znak &
  - ✓ `P` wstawi do kodu znak o numerze 80
  - ✓ `ñ` wstawi do kodu znak o numerze 241

Budowa dokumentu

- **Encje** https://symbl.cc/pl/html-entities/

Strzały

Symbole	HTML-kod	CSS-kod	Unicode	Mnemonika	Tytuł
↺	↺	\21BA	U+21BA	↺	Strzałka okrągła otwarta przeciwnie do ruchu wskazówek zegara
↻	↻	\21BB	U+21BB	↻	Strzałka okrągła otwarta w prawo
↼	↼	\21BC	U+21BC	↼	Harpun Leftwards z Barbem w górę
↽	↽	\21BD	U+21BD	↽	Harpun Leftwards z Barbem w dół
↾	↾	\21BE	U+21BE	↾	Harpun w górę z Barbem w prawo
↿	↿	\21BF	U+21BF	↿	Harpun w górę z Barbem Leftwards
⇀	⇀	\21C0	U+21C0	⇀	Harpun w prawo z Barbem w górę
⇁	⇁	\21C1	U+21C1	⇁	Harpun w prawo z Barbem w dół
⇂	⇂	\21C2	U+21C2	⇂	W dół Harpoon z Barbem w prawo

Przykład

- Zaprojektować dokument, który będzie przechowywał dane o zwycięzcach zawodów olimpijskich w następującej postaci:
  - ✓ Rok i miasto igrzysk.
  - ✓ Jakie były rozgrywane dyscypliny (nazwa danej dyscypliny).
  - ✓ Jakie były w nich konkurencje kobiet i mężczyzn.
  - ✓ Kto z jakiego państwa i jakim wynikiem zdobył jaki medal.

XML

- Węzeł igrzyska olimpijskie atrybuty: miasto, rok
  - Dyscyplina atrybut: nazwa
    - Płeć atrybut: nazwa
      - Konkurencja atrybut: nazwa
        - Złoto
          - Państwo encja: nazwa państwa
          - Nazwisko ecja: nazwisko zawodnika
          - Wynik encja: wynik
        - Srebro
          - Państwo encja: nazwa państwa
          - Nazwisko ecja: nazwisko zawodnika
          - Wynik encja: wynik
        - Braz
          - Państwo encja: nazwa państwa
          - Nazwisko ecja: nazwisko zawodnika
          - Wynik encja: wynik

XML

```xml
<?xml version="1.0" standalone="yes"?>
<igrzyskaOlimpijskie miasto="Ateny" rok="2004">
  <dyscyplina nazwa="Lekka Atletyka">
   <plec nazwa="K">
     <konkurencja nazwa="Bieg na 100 metrów">
      <zloto>
       <panstwo> Polska </panstwo>
       <nazwisko> Mala </nazwisko>
       <wynik> 11,3 </wynik>
      </zloto>
      <srebro>
       <panstwo> Czechy </panstwo>
       <nazwisko> Durek </nazwisko>
       <wynik> 14,7 </wynik>
      </srebro>
      <braz>
       <panstwo> Francja </panstwo>
       <nazwisko> Olga </nazwisko>
       <wynik> 17,9 </wynik>
      </braz>
     </konkurencja>
     <konkurencja nazwa="Bieg na 400 metrów">
      <zloto>
       <panstwo> Polska </panstwo>
       <nazwisko> Magda </nazwisko>
       <wynik> 44,2 </wynik>
      </zloto>
      <srebro>
       <panstwo> Czechy </panstwo>
       <nazwisko> Liza </nazwisko>
       <wynik> 49,9 </wynik>
      </srebro>
      <braz>
       <panstwo> Niemcy </panstwo>
       <nazwisko> Olga </nazwisko>
       <wynik> 58,2 </wynik>
      </braz>
     </konkurencja>
   </plec>
```

XML

```
<plec nazwa="M">
  <konkurencja nazwa="Bieg na 100 metrów">
   <zloto>
    <panstwo> Polska </panstwo>
    <nazwisko> Nowak </nazwisko>
    <wynik> 10,3 </wynik>
   </zloto>
   <srebro>
    <panstwo> Czechy </panstwo>
    <nazwisko> Barek </nazwisko>
    <wynik> 10,7 </wynik>
   </srebro>
   <braz>
    <panstwo> Niemcy </panstwo>
    <nazwisko> Otto </nazwisko>
    <wynik> 10,9 </wynik>
   </braz>
  </konkurencja>
```

```
  <konkurencja nazwa="Bieg na 400 metrów">
   <zloto>
    <panstwo> Polska </panstwo>
    <nazwisko> Barek </nazwisko>
    <wynik> 40,2 </wynik>
   </zloto>
   <srebro>
    <panstwo> Czechy </panstwo>
    <nazwisko> Borek </nazwisko>
    <wynik> 43,9 </wynik>
   </srebro>
   <braz>
    <panstwo> Niemcy </panstwo>
    <nazwisko> Olaf </nazwisko>
    <wynik> 50,2 </wynik>
   </braz>
  </konkurencja>
 </plec>
</dyscyplina>
```

XML


localhost/02%20-%20Olimpiada/b1.xml
This XML file does not appear to have any style information asso
tree is shown below.
<igrzyskaOlimpijskie miasto="Ateny" rok="2004">
<dyscyplina nazwa="Lekka Atletyka">
<plec nazwa="K">
...
</plec>
<plec nazwa="M">
<konkurencja nazwa="Bieg na 100 metrów">
...
</konkurencja>
<konkurencja nazwa="Bieg na 400 metrów">
...
</konkurencja>
</plec>
</dyscyplina>
<dyscyplina nazwa="Plywanie">
<plec nazwa="K">
<konkurencja nazwa="Delfin 100 metrów">
<zloto>
<panstwo> Polska </panstwo>
<nazwisko> Magda </nazwisko>
<wynik> 23 </wynik>
</zloto>
<srebro>

Kontrola struktury dokumentu

- Co robi
  - ✓ Które elementy mogą występować, w których.
  - ✓ Jakiego typu mogą być elementy.
  - ✓ Ile razy mogą wystąpić.
- W jaki sposób
  - ✓ DTD (Document Type Definition).
  - ✓ XML Schema.

DTD Włączenie kontroli poprawności

- W tym samym pliku

```
<!DOCTYPE nazwaDokumentu [
        Instrukcje opisujące
]>
```

- W pliku zewnętrznym (nazwa.dtd) . W pliku głównym dostawiamy jego dołączenie

```
<!DOCTYPE nazwaDokumentu SYSTEM|PUBLIC
  nazwaDTD>
```

Dla naszego przykładu

```
<!DOCTYPE igrzyskaOlimpijskie SYSTEM
  ”b1.dtd”>
```

Kontrola struktury dokumentu

- Definiowanie elementów

```
<!ELEMENT nazwa zawartość krotność>
```

  - ✓ Zawartość może być:
    - EMPTY — definicja elementu pustego.
    - ANY — dopuszcza każdą zawartość.
    - (#PCDATA) — element tekstowy.
    - (element) — potomny
    - (element1,element2 ..) — lista elementów
    - (element1|element2..) — jeden z podanych

Kontrola struktury dokumentu

- Definiowanie elementów

```
<!ELEMENT nazwa zawartość krotność>
```

  - Krotność wystąpień elementów:
    - (brak) Element ma wystąpić raz.
    - ? Element może wystąpić 0 lub jeden raz.
    - * Od zera do nieskończoności razy.
    - + Od 1 do nieskończoności razy.

Kontrola struktury dokumentu

- Definiowanie atrybutów

```
<!ATTLIST Element NazwaAtrybutu typ
  wystąpienie>
```

  - ✓ Typ może być
    - CDATA — Dowolna wartość tekstowa.
    - (opcja_1|opcja2..) — Dowolna opcja z listy.
    - NMTOKEN — Ma być nazwą XML
    - NMTOKENS — Ma być nazwą XML.
    - ID — Niepowtarzalna w całym w dokumencie.
    - IDREF lub IDREFS — Odwołanie do nazwy deklarowane atrybutem ID, lub listy atrybutów.

Kontrola struktury dokumentu

- Definiowanie atrybutów

```
<!ATTLIST Element NazwaAtrybutu typ
   wystąpienie>
```

  - ✓ Wystąpienie może być
    - wartość domyślna — Gdy nie podano będzie podstawiana.
    - #FIXED wartość — Ma być dokładnie ta.
    - #REQUIRED — Atrybut wymagany nie można go pominąć.
    - #IMPLIED — Atrybut nie ma wartości domyślne można go pominąć.

Kontrola struktury dokumentu

- Definiowanie skrótów

```
<!ENTITY nazwa wartosc>
```

Odwołanie się do skrótu następuje przez podanie **&nazwa;**

Kontrola struktury dokumentu

- Przykład

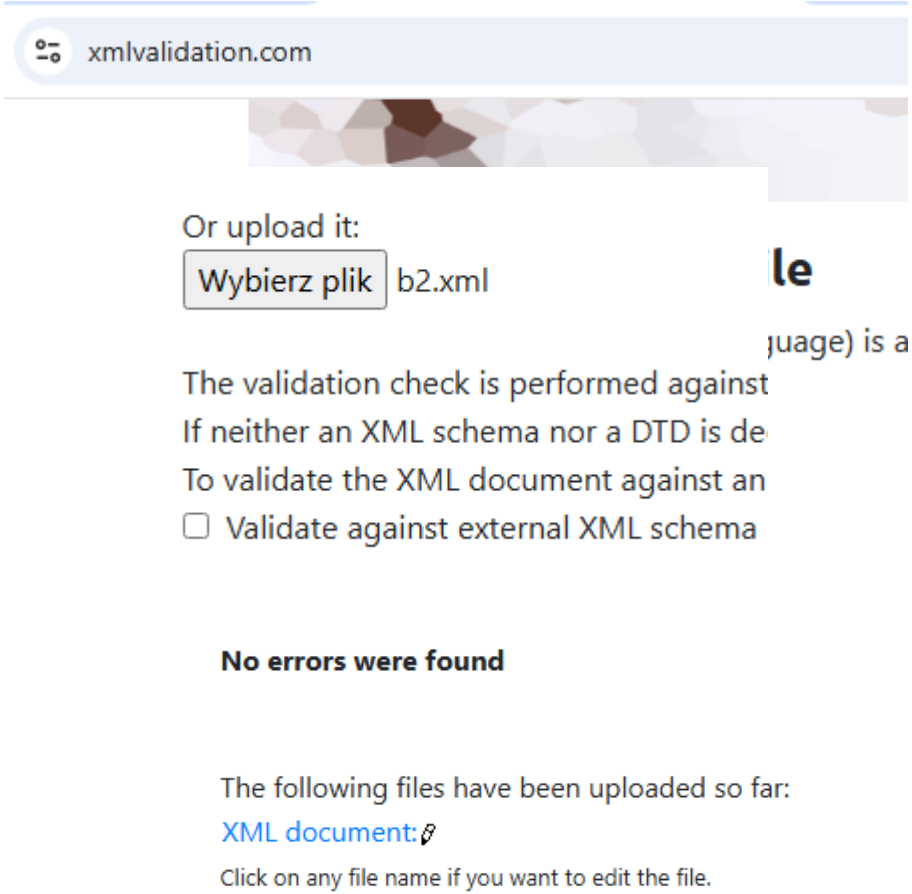
```
<!ELEMENT panstwo (#PCDATA)>
<!ELEMENT nazwisko (#PCDATA)>
<!ELEMENT wynik (#PCDATA)>
<!ELEMENT zloto (panstwo,nazwisko,wynik)>
<!ELEMENT srebro (panstwo,nazwisko,wynik)>
<!ELEMENT braz (panstwo,nazwisko,wynik)>
<!ELEMENT konkurencja (zloto,srebro,braz)>
<!ELEMENT plec (konkurencja+)>
<!ELEMENT dyscyplina (plec,plec?)>
<!ELEMENT igrzyskaOlimpijskie (dyscyplina+)>
<!ATTLIST igrzyskaOlimpijskie miasto CDATA #REQUIRED>
<!ATTLIST igrzyskaOlimpijskie rok CDATA #REQUIRED>
<!ATTLIST dyscyplina nazwa CDATA #REQUIRED>
<!ATTLIST plec nazwa (K|M) #REQUIRED>
<!ATTLIST konkurencja nazwa CDATA #REQUIRED>
```

```
<igrzyskaOlimpijskie miasto="Ateny" rok="2004">
  <dyscyplina nazwa="Lekka Atletyka">
    <plec nazwa="K">
      <konkurencja nazwa="Bieg na 100 metrów">
        <zloto>
          <panstwo> Polska </panstwo>
          <nazwisko> Mala </nazwisko>
          <wynik> 11,3 </wynik>
        </zloto>
        <srebro>
          <panstwo> Czechy </panstwo>
          <nazwisko> Durek </nazwisko>
          <wynik> 14,7 </wynik>
        </srebro>
        <braz>
          <panstwo> Francja </panstwo>
          <nazwisko> Olga </nazwisko>
          <wynik> 17,9 </wynik>
        </braz>
      </konkurencja>
```

Kontrola struktury dokumentu

- Przykład walidacja (https://www.xmlvalidation.com/)

```
<?xml version="1.0" standalone="yes"?>
<!DOCTYPE igrzyskaOlimpijskie [
<!ELEMENT panstwo (#PCDATA)>
<!ELEMENT nazwisko (#PCDATA)>
<!ELEMENT wynik (#PCDATA)>
<!ELEMENT zloto (panstwo,nazwisko,wynik)>
<!ELEMENT srebro (panstwo,nazwisko,wynik)>
<!ELEMENT braz (panstwo,nazwisko,wynik)>
<!ELEMENT konkurencja (zloto,srebro,braz)>
<!ELEMENT plec (konkurencja+)>
<!ELEMENT dyscyplina (plec,plec?)>
<!ELEMENT igrzyskaOlimpijskie (dyscyplina+)>
<!ATTLIST igrzyskaOlimpijskie miasto CDATA #REQUIRED>
<!ATTLIST igrzyskaOlimpijskie rok CDATA #REQUIRED>
<!ATTLIST dyscyplina nazwa CDATA #REQUIRED>
<!ATTLIST plec nazwa (K|M) #REQUIRED>
<!ATTLIST konkurencja nazwa CDATA #REQUIRED>
]>
<igrzyskaOlimpijskie miasto="Ateny" rok="2004">
  <dyscyplina nazwa="Lekka Atletyka">
   <plec nazwa="K">
     <konkurencja nazwa="Bieg na 100 metrów">
      <zloto>
       <panstwo> Polska </panstwo>
       <nazwisko> Mala </nazwisko>
       <wynik> 11,3 </wynik>
      </zloto>
      <srebro>
       <panstwo> Czechy </panstwo>
       <nazwisko> Durek </nazwisko>
       <wynik> 14,7 </wynik>
```



Oglądanie danych XML

- Arkusze stylów Arkusze CSS składają się ze zbioru reguł opisujących wygląd poszczególnych elementów w przeglądarce.

```
<?xml-stylesheet type="text/css"
  href="arkusz.css"?>
```

Oglądanie danych XML

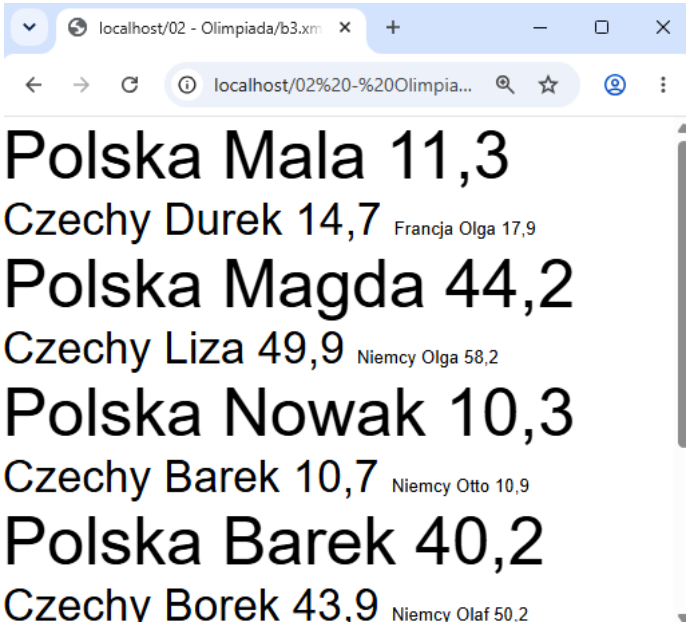
- Wykaz medalistów

b3.css

```
zloto{
    font-family: "Courier", sans-serif;
    font-size:30pt;  }
srebro{
    font-family: "Ariel", sans-serif;
    font-size:20pt; }
braz{
    font-family: "Gill Sans", sans-serif;
     font-size:8pt;  }
```

b3.xml

```
<?xml version="1.0" standalone="yes"?>
<?xml-stylesheet type="text/css" href="b3.css"?>
<!DOCTYPE igrzyskaOlimpijskie [
<!ELEMENT panstwo (#PCDATA)>
<!ELEMENT nazwisko (#PCDATA)>
<!ELEMENT wynik (#PCDATA)>
<!ELEMENT zloto (panstwo,nazwisko,wynik)>
<!ELEMENT srebro (panstwo,nazwisko,wynik)>
<!ELEMENT braz (panstwo,nazwisko,wynik)>
<!ELEMENT konkurencja (zloto,srebro,braz)>
```



Oglądanie danych XML

- Język XSL
  - ✓ Język XSL (który sam jest aplikacją XML) pozwala przekształcać dane XML na podstawie czegoś w rodzaju szablonu.
  - ✓ Korzystając z tego narzędzia można dane przegrupowywać i wybierać. Można je także przygotować do wizualnego przeglądania.
  - ✓ Na standard XSL składa się:
    - ❑ Język samych przekształceń (XSLT).
    - ❑ Język opisu graficznego (obiekty formatujące FO).

Język XSL

- Arkusze stylów XSL mogą być osobnymi plikami, które za pomocą odpowiedniej deklaracji w prologu przypisuje się dokumentom z danymi. Oczywiście jednego arkusza stylów można używać do przekształceń wielu plików z danymi tworzącymi tę samą strukturę.

```
<?xml-stylesheet type=”text/xml”
 href=”default.xsl”?>
```

Język XSL – składnia języka

- W celu odróżnienia polecenia XSLT, muszą być one poprzedzane przedrostkiem.

  `xsl:`

- Główny element każdego arkusza XSL.

  `xsl:stylesheet`

- Zawiera on dwa atrybuty
  - ✓ `version` aktualnie „1.0”.
  - ✓ `xmlns:xsl` odpowiada za lokalizację dokumentu z przestrzenią nazw dla XSLT.

Język XSL – składnia języka

- Wybierają one określone przez atrybut **match** węzły przetwarzanego dokumentu źródłowego. Z wyselekcjonowanych w ten sposób węzłów można pobrać dane.

```
xsl:template
```

- Parser zatrzyma obsługę bieżącej sekcji **xsl:template** i zacznie szukać nowej, odpowiadającej węzłom-dzieciom bieżącego taga. Kiedy wszystkie węzły-dzieci już przetworzy, to wróci do pierwotnej sekcji **xsl:template**. Zamiast wskazania na pod węzły można użyć frazy **select**

```
<xsl:apply-templates
    select=”sciezka do wezla”/>
```

Język XSL – przykład

c1.xsl

Plik Edytuj Wyświetl

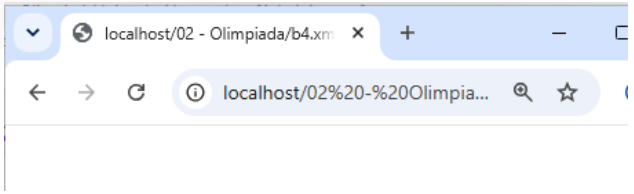
```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
 <xsl:template match="/">
  <html>

  </html>
 </xsl:template>
</xsl:stylesheet>
```

```
<html>
</html>
```

b4.xml

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <?xml-stylesheet type="text/xsl" href="c2.xsl"?>
3
4  <!DOCTYPE igrzyskaOlimpijskie [
5  <!ELEMENT panstwo (#PCDATA)>
6  <!ELEMENT nazwisko (#PCDATA)>
7  <!ELEMENT wynik (#PCDATA)>
```



Kolejne slajdy c2, c3 ..

Język XSL – składnia języka

- Pobiera wartości z dokumentu. Posiada jeden parametr - `select`. Określa on, co ma zostać pobrane.

  `xsl:value-of`

  - ✓ `.` Bieżący węzeł
  - ✓ `@` Wartość atrybutu
  - ✓ `/` Kiedy ścieżka zawiera więcej niż jeden element (czy np. atrybut i podelement), oddzielamy wszystko znakami „/”

Język XSL – przykład Złoci medaliści.



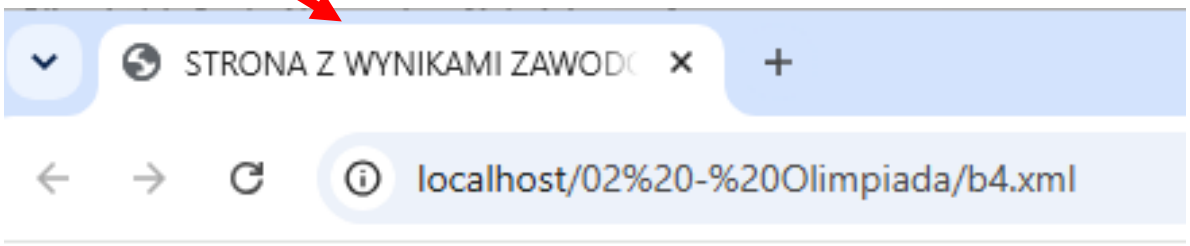
Język XSL – przykład Złoci medaliści.

- Korzeń → Ramówka strony
- igrzyskaOlimpijskie → **O to dane na temat medalistow zawodow Olimpijskich rozegranych w Ateny w roku 2004**
- Dyscyplina → **Dyscyplina: Lekka Atletyka**
- Plec
- Konkurencja → **Konkurencje: Bieg na 100 metrów K**

ZLOTO DLA : Mala Z KRAJU : Polska WYNIK : 11,3

Język XSL – przykład Złoci medaliści.

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
 <xsl:template match="/">
 <html>
  <title>
    STRONA Z WYNIKAMI ZAWODOW
 </title>
 <body>
  <xsl:apply-templates/>
 </body>
 </html>
 </xsl:template>
```



Język XSL – przykład Złoci medaliści.

```
<xsl:template match="igrzyskaOlimpijskie">
 <p>
  <center> <h1>
    O to dane na temat medalistow zawodow Olimpijskich rozegranych w
     <xsl:value-of select="@miasto"/> w roku <xsl:value-of select="@rok"/>
  </h1></center>
  <H2> Uzyskano nastepujace wyniki w dyscyplinach :</H2>
 </p>
 <xsl:apply-templates/>
</xsl:template>
```



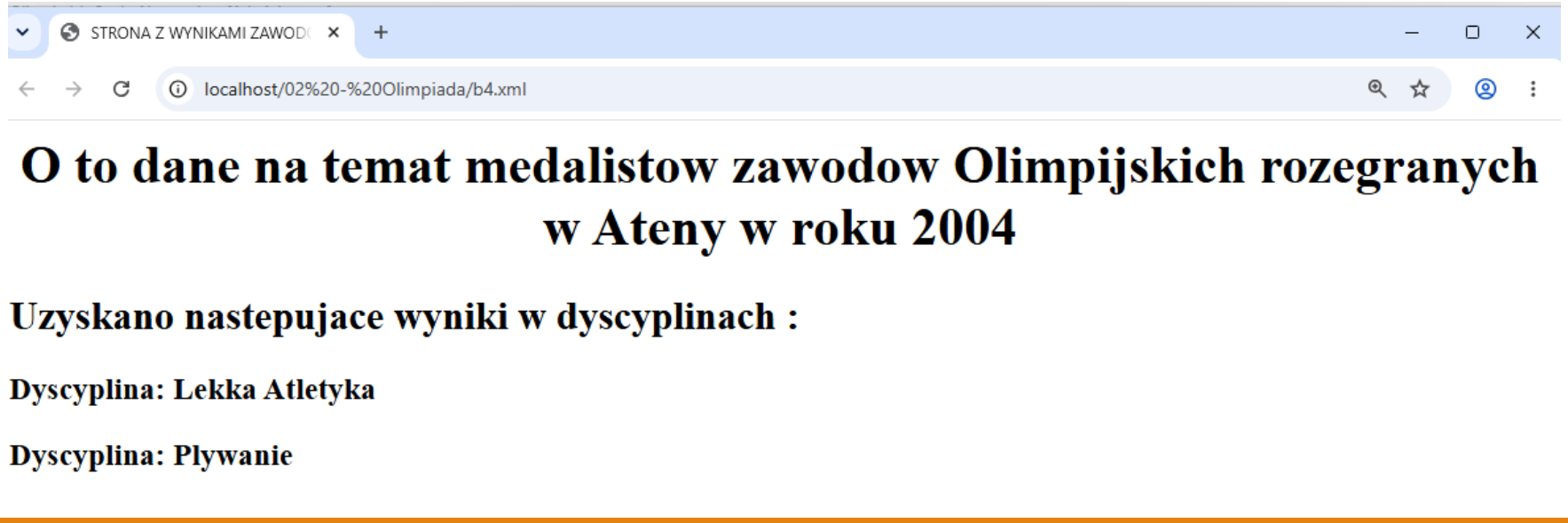
O to dane na temat medalistow zawodow Olimpijskich rozegranych w Ateny w roku 2004

Uzyskano nastepujace wyniki w dyscyplinach :

Język XSL – przykład Złoci medaliści.

```
<xsl:template match="dyscyplina">
  <p><h3> Dyscyplina: <xsl:value-of select="@nazwa"/> </h3></p>
 <xsl:apply-templates/>
</xsl:template>

<xsl:template match="plec">
 <xsl:apply-templates/>
</xsl:template>
```



Język XSL – przykład Złoci medaliści.

```
<xsl:template match="konkurencja">
 <p><h3> Konkurencje: <xsl:value-of select="@nazwa"/>
   <xsl:value-of select="../@nazwa"/>
 </h3></p>
   <BR></BR>
    ZLOTO DLA : <xsl:value-of select="zloto/nazwisko"/>
      Z KRAJU : <xsl:value-of select="zloto/panstwo"/>
        WYNIK : <xsl:value-of select="zloto/wynik"/>
</xsl:template>
```

Konkurencje: Bieg na 100 metrów K

ZLOTO DLA : Mala Z KRAJU : Polska WYNIK : 11,3

Konkurencje: Bieg na 400 metrów K

Normalizacja danych

- Uczestnicy (plik n1.xml)
  - Państwo atrybut: nazwa
    - Zawodnik **atrybut: id** atrybut: PlecZ
      - Nazwisko
      - Wiek
      - …

Normalizacja danych

- Uczestnicy (plik n1.xml)

n1.dtd

```
<!ELEMENT Nazwisko (#PCDATA)>
<!ELEMENT Wiek (#PCDATA)>
<!ELEMENT Zawodnik (Nazwisko,Wiek)>
<!ELEMENT Panstwo (Zawodnik+)>
<!ELEMENT Uczestnicy (Panstwo+)>
<!ATTLIST Panstwo nazwa CDATA #REQUIRED>
<!ATTLIST Zawodnik PlecZ (K|M) #REQUIRED>
<!ATTLIST Zawodnik id ID #REQUIRED>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<Uczestnicy>
  <Panstwo nazwa="Polska">
    <Zawodnik id="Z01" PlecZ="M">
      <Nazwisko>Kowalski</Nazwisko>
      <Wiek>23</Wiek>
    </Zawodnik>
    <Zawodnik id="Z02" PlecZ="M">
      <Nazwisko>Nowak</Nazwisko>
      <Wiek>42</Wiek>
    dnik>
    nik id="Z03" PlecZ="K">
    wisko>Duda</Nazwisko>
    k>31</Wiek>
    dnik>
    o>
     nazwa="Francja">
    nik id="Z04" PlecZ="M">
    wisko>Olaf</Nazwisko>
    k>22</Wiek>
    dnik>
    nik id="Z05" PlecZ="M">
    wisko>Buda</Nazwisko>
    k>44</Wiek>
    </Zawodnik>
    <Zawodnik id="Z06" PlecZ="K">
      <Nazwisko>Graf</Nazwisko>
      <Wiek>23</Wiek>
    </Zawodnik>
  </Panstwo>
</Uczestnicy>
```

Normalizacja danych

- Zawody (plik n2.xml)
  - ✓ Dyscyplina — atrybut: nazwa
    - ❑ Plec — atrybut: nazwa
      - Konkurencja — **atrybut: id** atrybut: nazwa
      - ….

Normalizacja danych

- Zawody (plik n2.xml)



n2.dtd

```
<!ELEMENT Konkurencja (#PCDATA)>
<!ELEMENT Plec  (Konkurencja+)>
<!ELEMENT Dyscyplina (Plec+)>
<!ELEMENT Zawody (Dyscyplina+)>
<!ATTLIST Konkurencja id ID #REQUIRED>
<!ATTLIST Konkurencja Nazwa CDATA  #REQUIRED>
<!ATTLIST Plec Nazwa (K|M) #REQUIRED>
<!ATTLIST Dyscyplina Nazwa CDATA #REQUIRED>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<Zawody>
  <Dyscyplina Nazwa="Lekka Atletyka">
    <Plec Nazwa="K">
      <Konkurencja id="K01" Nazwa="Bieg na 100 metrów">
      </Konkurencja>
      <Konkurencja id="K02" Nazwa="Bieg na 400 metrów">
      </Konkurencja>
    </Plec>
    <Plec Nazwa="M">
      <Konkurencja id="K03" Nazwa="Bieg na 100 metrów">
      </Konkurencja>
      ncja id="K04" Nazwa="Bieg na 400 metrów">
      encja>

      a>
       Nazwa="Plywanie">
      ="K">
      ja id="K05" Nazwa="Delfin 100 metrów">
      cja>
      ja id="K06" Nazwa="Na wznak 100 metrów">
      cja>

    </Dyscyplina>
</Zawody>
```

Normalizacja danych

- igrzyskaOlimpijskie

  atrybut: miasto atrybut: rok

  - ✓ Uczestnicy
  - ✓ Zawody
  - ✓ Wynik
    - Zloto atrybut: idz atrybut: idk
    - Srebro atrybut: idz atrybut: idk
    - Braz atrybut: idz atrybut: idk

Normalizacja danych

n2.dtd | n.xml

```
<?xml version="1.0" encoding="UTF-8" ?>
<!DOCTYPE igrzyskaOlimpijskie SYSTEM "n.dtd">
<igrzyskaOlimpijskie miasto="Ateny" rok="2004">
 &ucz;
 &zaw;
 <wynik>
  <zloto  idz="Z01" idk="K01">20,3</zloto>
  <srebro idz="Z01" idk="K01">30,4</srebro>
  <braz   idz="Z01" idk="K01">40,3</braz>
 </wynik>
 <wynik>
  <zloto  idz="Z03" idk="K02">20,3</zloto>
  <srebro idz="Z04" idk="K02">30,4</srebro>
  <braz   idz="Z05" idk="K02">40,3</braz>
 </wynik>
 <wynik>
  <zloto  idz="Z06" idk="K03">20,3</zloto>
  <srebro idz="Z01" idk="K03">30,4</srebro>
  <braz   idz="Z02" idk="K03">40,3</braz>
 </wynik>
 <wynik>
  <zloto  idz="Z01" idk="K04">20,3</zloto>
  <srebro idz="Z02" idk="K04">30,4</srebro>
  <braz   idz="Z02" idk="K04">40,3</braz>
 </wynik>
</igrzyskaOlimpijskie>
```

n2.dtd | n.xml | n.dtd

```
<!ENTITY  ucz  SYSTEM "n1.xml">
<!ENTITY  zaw  SYSTEM "n2.xml">
<!ENTITY % ucz_d  SYSTEM "n1.dtd">
<!ENTITY % zaw_d  SYSTEM "n2.dtd">
%ucz_d;
%zaw_d;
<!ELEMENT zloto (#PCDATA)>
<!ELEMENT srebro (#PCDATA)>
<!ELEMENT braz (#PCDATA)>
<!ELEMENT wynik (zloto,srebro,braz)>
<!ELEMENT igrzyskaOlimpijskie (Uczestnicy,Zawody,wynik+)>
<!ATTLIST igrzyskaOlimpijskie miasto CDATA #REQUIRED>
<!ATTLIST igrzyskaOlimpijskie rok CDATA #REQUIRED>
<!ATTLIST zloto idz IDREF #REQUIRED>
<!ATTLIST zloto idk IDREF #REQUIRED>
<!ATTLIST srebro idz IDREF #REQUIRED>
<!ATTLIST srebro idk IDREF #REQUIRED>
<!ATTLIST braz idz IDREF #REQUIRED>
<!ATTLIST braz idk IDREF #REQUIRED>
```

Normalizacja danych



Brak danych z dołączanych plików



Normalizacja danych

No errors were found

The following files have been uploaded so far:

XML document:

n.dtd

n2.xml

n1.xml

n2.dtd

n1.dtd

Click on any file name if you want to edit the file.

Język XSL – zmienne

- Definicja zmiennej

```
<xsl:variable name="naz">
    wartosc_zmiennej
</xsl:variable>
```

- Aby odwołać się do zmiennej poprzedzamy jej nazwę znakiem `$`.
- Zmienne dostępne są w obrębie bloku w którym były zadeklarowane

Język XSL – Adresacja węzłów

- Określenie położenia węzła w drzewie polega na podaniu do niego ścieżki
  - ✓ Bezwzględnej
  - ✓ Względnej
- Pojawienie się w ścieżce nawiasu klamrowego [] oznacza budowanie warunku

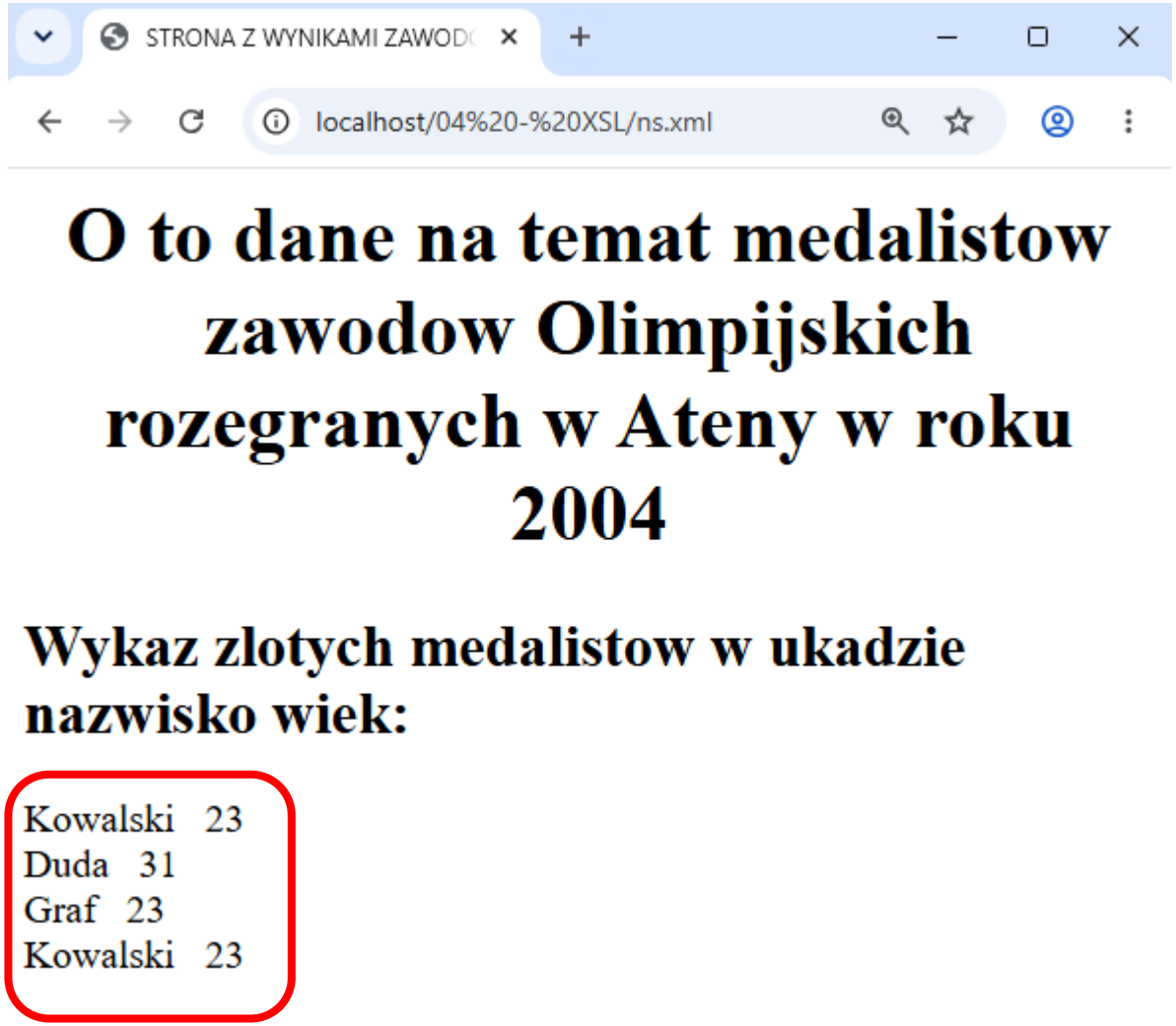
```
/igrzyskaOlimpijskie/uczestnicy/panstwo/zawodnik[@nazwisko='Kowalski']
```

- Warunki można łączyć za pomocą instrukcji `and` lub `or`.

Język XSL – Adresacja węzłów

- Dodatkowo można używać funkcji
  - `last()` Ostatni.
  - `position()` Węzeł o podanym numerze position()=5.
  - `count(zbiór)` Zlicza węzły spełniające warunek.
  - **`id(obiek)`** Węzła wskazywanego przez obiekt

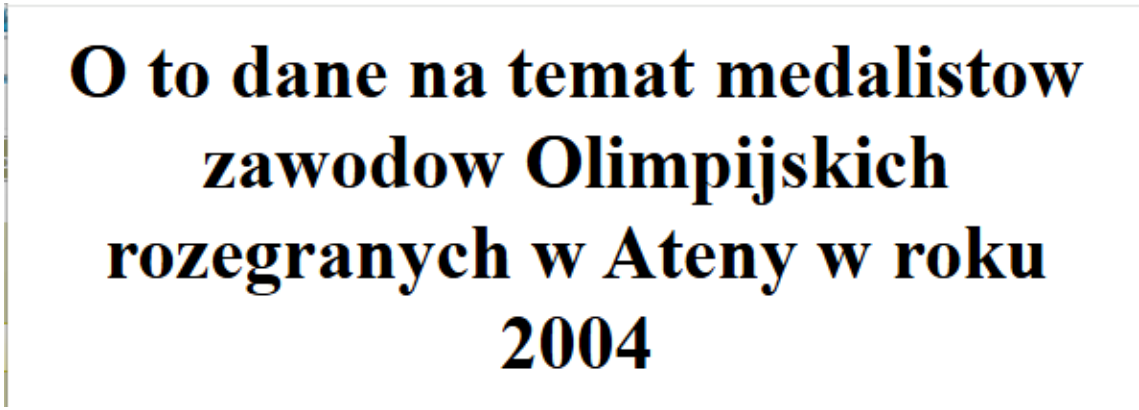
Język XSL – przykład Wiek złotych medalistów. (ns1.xsl)



Język XSL – przykład Wiek złotych medalistów.

- Korzeń
- igrzyskaOlimpijskie

Ramówka strony



- Wynik

Kowalski 23
Duda 31
Graf 23
Kowalski 23

Język XSL – przykład Wiek złotych medalistów.

- Korzeń

ns1.xsl

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
 <xsl:template match="/">
 <html>
  <title>
    STRONA Z WYNIKAMI ZAWODOW
 </title>
 <body>
   <xsl:apply-templates/>
 </body>
 </html>
 </xsl:template>
```

Język XSL – przykład Wiek złotych medalistów.

- IgrzyskaOlimoijskie

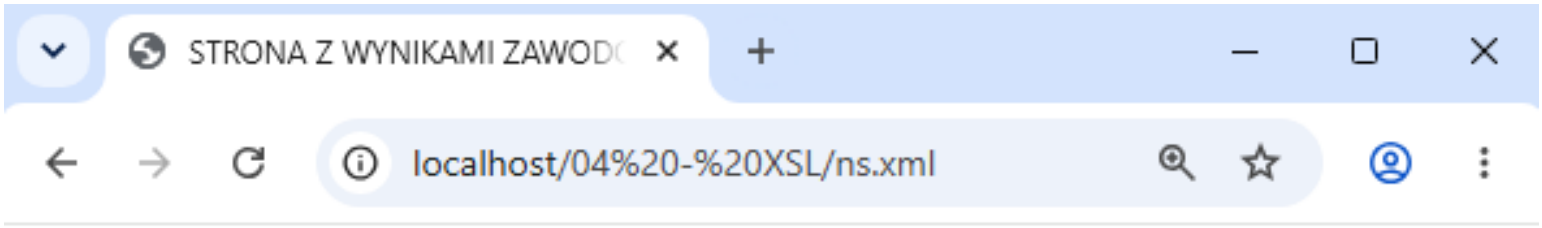
```
<xsl:template match="igrzyskaOlimpijskie">
 <p>
  <center> <h1>
    O to dane na temat medalistow zawodow Olimpijskich rozegranych w
     <xsl:value-of select="@miasto"/> w roku <xsl:value-of select="@rok"/>
  </h1></center>
  <H2> Wykaz zlotych medalistow  w ukadzie nazwisko wiek:</H2>
 </p>
  <xsl:apply-templates select="wynik"/>
</xsl:template>
```

Język XSL – przykład Wiek złotych medalistów.

- Wynik

```
<xsl:template match="wynik">
 <xsl:variable name="idZa">
   <xsl:value-of select="zloto/@idz" />
 </xsl:variable>
 <xsl:value-of
    select="/igrzyskaOlimpijskie/Uczestnicy/Panstwo/Zawodnik[@id=$idZa]/Nazwisko"/> 
 <xsl:value-of
     select="/igrzyskaOlimpijskie/Uczestnicy/Panstwo/Zawodnik[@id=$idZa]/Wiek"/> <BR/>
</xsl:template>
</xsl:stylesheet>
```

Język XSL – przykład Tabelaryczny wykaz złotych medalistów (ns2.xsl)



Tabelarczny wykaz zlotych medalistow Ateny w roku 2004

Dyscyplina	Konkurencja	Plec	Nazwisko	Panstwo
Bieg na 100 metrów	Lekka Atletyka	Kobieta	Kowalski	Polska
Bieg na 400 metrów	Lekka Atletyka	Kobieta	Duda	Polska
Bieg na 100 metrów	Lekka Atletyka	Mezczyzna	Graf	Francja
Bieg na 400 metrów	Lekka Atletyka	Mezczyzna	Kowalski	Polska

Język XSL – przykład Tabelaryczny wykaz złotych medalistów

- Korzeń — Ramówka strony
- igrzyskaOlimpijskie — Napis, nagłówek tabeli
- Wynik — Ustalenie **idz idk**
- Konkurencja — Nazwa konkurencji, dyscypliny pleć
- Zawodnik — Nazwisko zawodnika, nazwa państwa

Język XSL – przykład Tabelaryczny wykaz złotych medalistów – węzeł główny

ns2.xsl

```
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
 <xsl:template match="/">
 <html>
  <title>
    STRONA Z WYNIKAMI ZAWODOW
 </title>
 <body>
   <xsl:apply-templates/>
 </body>
 </html>
 </xsl:template>
```

Język XSL – przykład Tabelaryczny wykaz złotych medalistów – węzeł igrzyskaOlimpijskie

```
<xsl:template match="igrzyskaOlimpijskie">
 <p>
  <center> <h1>
   Tabelarczny wykaz zlotych medalistow
     <xsl:value-of select="@miasto"/> w roku <xsl:value-of select="@rok"/>
  </h1></center>
 </p>
  <table border="frame">
  <tr> <th>Dyscyplina</th>
       <th>Konkurencja</th>
       <th>Plec</th>
       <th>Nazwisko</th>
       <th>Panstwo</th>
  </tr>
  <xsl:apply-templates select="wynik"/>
  </table>
</xsl:template>
```



Język XSL – przykład Tabelaryczny wykaz złotych medalistów – węzeł wynik

```
<xsl:template match="wynik">
 <xsl:variable name="idZa">
   <xsl:value-of select="zloto/@idz" />
 </xsl:variable>
 <xsl:variable name="idKo">
   <xsl:value-of select="zloto/@idk" />
 </xsl:variable>
 <tr>
   <xsl:apply-templates select="id($idKo)"/>
   <xsl:apply-templates select="id($idZa)"/>
 </tr>
</xsl:template>
```

Język XSL – przykład Tabelaryczny wykaz złotych medalistów – węzeł Konkurencja węzeł zawodnik

```
<xsl:template match="Konkurencja">
  <td> <xsl:value-of select="@Nazwa" /></td>
  <td> <xsl:value-of select="../../@Nazwa" /></td>
  <td> <xsl:value-of select="../@Nazwa" /></td>
</xsl:template>
<xsl:template match="Zawodnik">
  <td> <xsl:value-of select="Nazwisko" /></td>
  <td> <xsl:value-of select="../@nazwa" /></td>
</xsl:template>
```

Bieg na 100 metrów	Lekka Atletyka	K	Kowalski	Polska
Bieg na 400 metrów	Lekka Atletyka	K	Duda	Polska
Bieg na 100 metrów	Lekka Atletyka	M	Graf	Francja
Bieg na 400 metrów	Lekka Atletyka	M	Kowalski	Polska

Język XSL – instrukcje warunkowe

- Instrukcja if

```
<xsl:if test="zloto/panstwo='Polska'">
  WYNIK :
</xsl:if>
```

- Instrukcja choose

```
<xsl:choose>
  <xsl:when test="@nazwisko='Kowalski'">
      Kowalski
  </xsl:when>
  <xsl:otherwise>
      Inny
  </xsl:otherwise>
</xsl:choose>
```

Język XSL – przykład Tabelaryczny wykaz złotych medalistów - – pełna nazwa płci zawodnika (n3.xsl)

```
<xsl:template match="Konkurencja">
  <td> <xsl:value-of select="@Nazwa" /></td>
  <td> <xsl:value-of select="../../@Nazwa" /></td>
  <td> <xsl:value-of select="../@Nazwa" /></td>
</xsl:template>
```

Tak było



ırencja	Plec	Nazwisk
Atletyka	K	Kowalski
Atletyka	K	Duda
Atletyka	M	Graf
Atletyka	M	Kowalski

Język XSL – przykład Tabelaryczny wykaz złotych medalistów - – pełna nazwa płci zawodnika

```
<xsl:template match="Konkurencja">
  <td> <xsl:value-of select="@Nazwa" /></td>
  <td> <xsl:value-of select="../../@Nazwa" /></td>
  <td> <xsl:if test="../@Nazwa='K'">Kobieta </xsl:if>
       <xsl:if test="../@Nazwa='M'">Mezczyzna </xsl:if>
  </td>
</xsl:template>
```



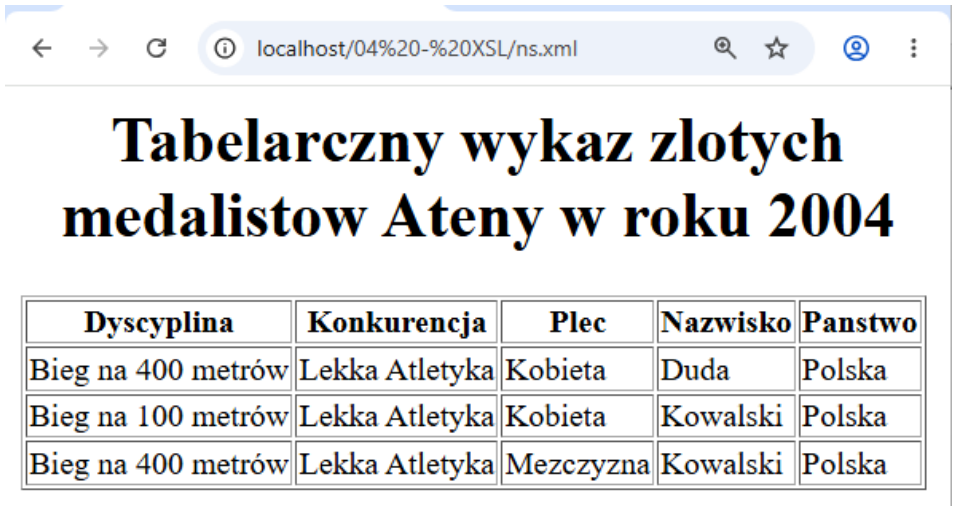
Język XSL – przykład Tabelaryczny wykaz złotych medalistów – tylko z polski (n4.xsl)

```
<xsl:template match="wynik">
 <xsl:variable name="idZa">
   <xsl:value-of select="zloto/@idz" />
 </xsl:variable>
 <xsl:variable name="idKo">
   <xsl:value-of select="zloto/@idk" />
 </xsl:variable>
 <tr>
  <xsl:apply-templates select="id($idKo)"/>
  <xsl:apply-templates select="id($idZa)"/>
 </tr>
</xsl:template>
```

Tak było

Język XSL – przykład Tabelaryczny wykaz złotych medalistów – tylko z polski

```
<xsl:template match="wynik">
 <xsl:variable name="idZa">
   <xsl:value-of select="zloto/@idz" />
 </xsl:variable>
 <xsl:variable name="pan">
   <xsl:value-of select="/igrzyskaOlimpijskie/Uczestnicy/Panstwo/Zawodnik[@id=$idZa]/../@nazwa" />
 </xsl:variable>
 <xsl:if test="$pan='Polska'">
   <xsl:variable name="idKo">
     <xsl:value-of select="zloto/@idk" />
   </xsl:variable>
   <tr>
     <xsl:apply-templates select="id($idKo)"/>
     <xsl:apply-templates select="id($idZa)"/>
   </tr>
 </xsl:if>
</xsl:template>
```



Dyscyplina	Konkurencja	Plec	Nazwisko	Panstwo
Bieg na 400 metrów	Lekka Atletyka	Kobieta	Duda	Polska
Bieg na 100 metrów	Lekka Atletyka	Kobieta	Kowalski	Polska
Bieg na 400 metrów	Lekka Atletyka	Mezczyzna	Kowalski	Polska

Język XSL – sortowanie danych

- Instrukcja for-each

```
<xsl:for-each select=”węzeł”>
      <xsl:sort select=”atrybut/węzeł”/>
</xsl:for-each>
```

Język XSL – przykład Tabelaryczny wykaz złotych medalistów – tylko z polski posortowani alfabetycznie (n5.xsl)

```
<xsl:template match="igrzyskaOlimpijskie">
 <p>
  <center> <h1>
   Tabelarczny wykaz zlotych medalistow
     <xsl:value-of select="@miasto"/> w roku <xsl:value-of select="@rok"/>
  </h1></center>
 </p>
  <table border="frame">
  <tr> <th>Dyscyplina</th>
       <th>Konkurencja</th>
       <th>Plec</th>
       <th>Nazwisko</th>
       <th>Panstwo</th>
  </tr>
  <xsl:apply-templates select="wynik"/>
  </table>
</xsl:template>
```

Tak było

Język XSL – przykład Tabelaryczny wykaz złotych medalistów – tylko z polski posortowani alfabetycznie

```
<xsl:for-each select="Uczestnicy/Panstwo/Zawodnik">
  <xsl:sort select="Nazwisko" />
    <xsl:variable name="idZa">
       <xsl:value-of select="@id" />
   </xsl:variable>
   <xsl:for-each select="/igrzyskaOlimpijskie/wynik/zloto[@idz=$idZa]">
    <xsl:variable name="pan">
     <xsl:value-of select="/igrzyskaOlimpijskie/Uczestnicy/Panstwo/Zawodnik[@id=$idZa]/../@nazwa" />
    </xsl:variable>
    <xsl:if test="$pan='Polska'">
     <xsl:variable name="idKo">
      <xsl:value-of select="@idk" />
     </xsl:variable>
     <tr>
      <xsl:apply-templates select="id($idKo)"/>
      <xsl:apply-templates select="id($idZa)"/>
     </tr>
     </xsl:if>
   </xsl:for-each>
</xsl:for-each>
```

Jest to niewielki zakres możliwości XML

Obsługa XML z poziomu języka PHP

- Wczytuje i konwertuje dokument XML do obiektu.

```
object simplexml_load_file(string $filename
 [,string $class_name [,int $options]])
```

Parametry:

- ✓ $filename Nazwa pliku XML
- ✓ $class_name Potomna SimpleXMLElement
- ✓ $options Sterowanie sprawdzania DTD.

- Wczytuje string i konwertuje go do obiektu XML.

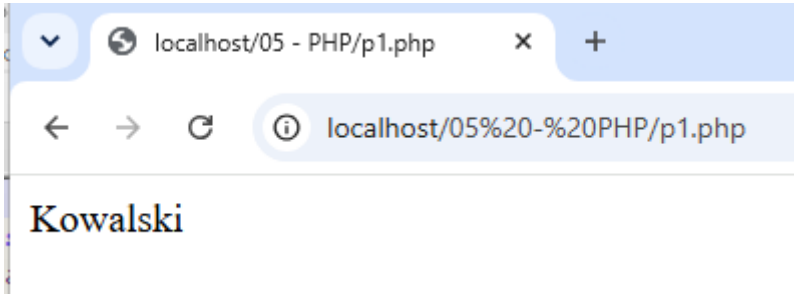
```
object simplexml_load_string ( string $data
 [,string $class_name [, int $options]])
```

Obsługa XML z poziomu języka PHP - metody

- Metody przechwytująca umożliwiająca odczytanie tekstowej reprezentacji każdego elementu XML (CDATA)

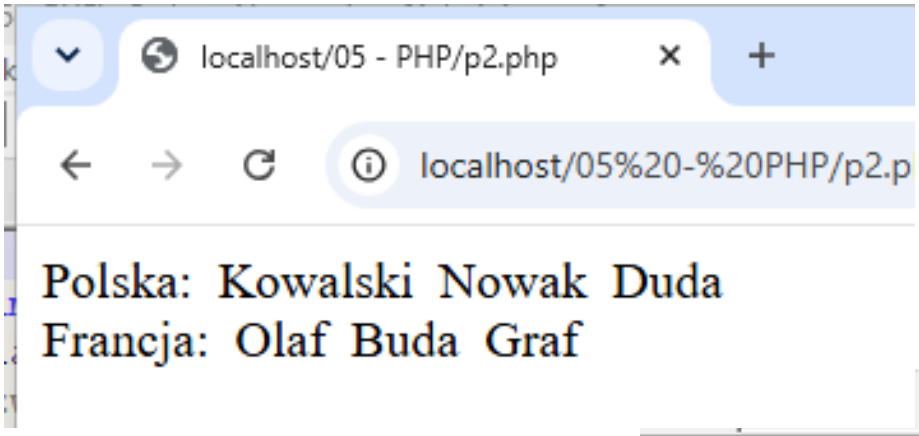
  __toString,

```php
<?php
    $olimpiady = simplexml_load_file('ns.xml');
    print $olimpiady->Uczestnicy[0]->Panstwo[0]->Zawodnik[0]->Nazwisko;
?>
```



Obsługa XML z poziomu języka PHP - metody

- Podać dane w układzie Panstwo Nazwisko



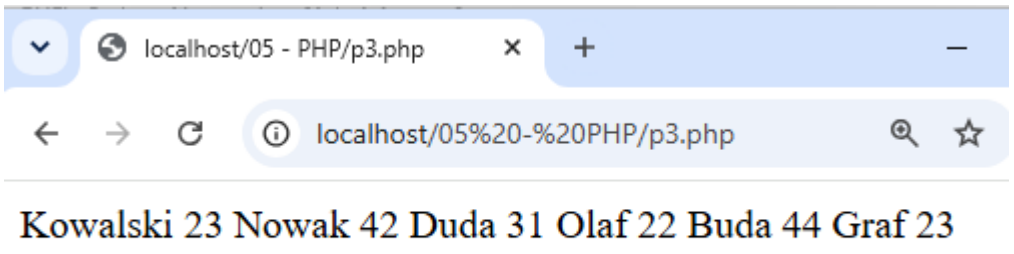

```php
<?php
   $olimpiady = simplexml_load_file('ns.xml');
   foreach($olimpiady->Uczestnicy->Panstwo as $pan)
   {echo $pan['nazwa'].": " ;
    foreach($pan as $zaw)
    {echo $zaw->Nazwisko." ";}
     echo "<BR/>";
   }
?>
```

Obsługa XML z poziomu języka PHP - metody

- Metoda wyświetla dane w układzie struktury XML

  asXML()

```php
<?php
  $olimpiady = simplexml_load_file('ns.xml');
  $ucz=$olimpiady->Uczestnicy ;
  print $ucz->asXML();
  file_put_contents('plik.txt', $ucz->asXML());
?>
```



plik.txt

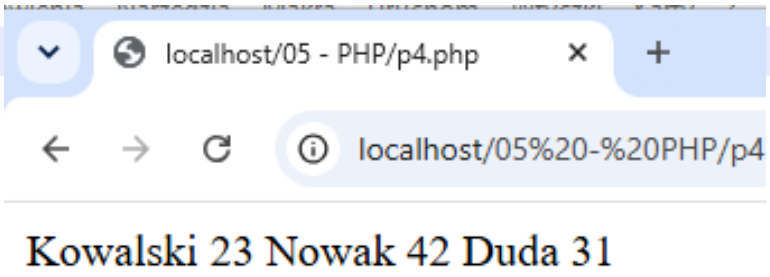
```
<Uczestnicy>
  <Panstwo nazwa="Polska">
    <Zawodnik id="Z01" PlecZ="M">
      <Nazwisko>Kowalski</Nazwisko>
      <Wiek>23</Wiek>
    </Zawodnik>
    <Zawodnik id="Z02" PlecZ="M">
      <Nazwisko>Nowak</Nazwisko>
      <Wiek>42</Wiek>
    </Zawodnik>
    <Zawodnik id="Z03" PlecZ="K">
      <Nazwisko>Duda</Nazwisko>
      <Wiek>31</Wiek>
    </Zawodnik>
  </Panstwo>
  <Panstwo nazwa="Francja">
    <Zawodnik id="Z04" PlecZ="M">
      <Nazwisko>Olaf</Nazwisko>
      <Wiek>22</Wiek>
    </Zawodnik>
    <Zawodnik id="Z05" PlecZ="M">
      <Nazwisko>Buda</Nazwisko>
      <Wiek>44</Wiek>
    </Zawodnik>
    <Zawodnik id="Z06" PlecZ="K">
      <Nazwisko>Graf</Nazwisko>
      <Wiek>23</Wiek>
    </Zawodnik>
  </Panstwo>
</Uczestnicy>
```

Obsługa XML z poziomu języka PHP - metody

- Dzięki tej metodzie możemy wybrać dowolny węzeł Wynikiem takich zapytań jest zawsze tablica obiektów typu SimpleXMLElement.

xpath()

```php
<?php
  $olimpiady = simplexml_load_file('ns.xml');
  $ucz=$olimpiady->xpath('Uczestnicy/Panstwo[@nazwa="Polska"]') ;
  print $ucz[0]->asXML();
?>
```



Obsługa XML z poziomu języka PHP - metody

- Zwraca nazwę węzła XML

```
getName()
```

- Dodaje atrybut w węźle

```
addAttribute (string $name,  string $value)
```

- Dodaje element i zwraca go pod swoją nazwą

```
addChild ( string $name [, string $value
  [, string $namespace]] )
```

Obsługa XML z poziomu języka PHP - metody

```php
<?php
  $sxe=simplexml_load_string('<?xml version="1.0"
    encoding="UTF-8"?><studenci></studenci>');
  $sxe->addAttribute('Grupa','314A');
  $s=$sxe->addChild('student');
  $s->addChild('nazwisko','Kowalski');
  $s->addChild('imie','Jan');
  $p=$s->addChild('przedmiot');
  $n=$p->addChild('Nazwa','PAI');
  $n=$p->addChild('Nazwa','FIZYKA');
  $fp = fopen("plik.xml", "w");
  fputs($fp,$sxe->asXML());
?>
```

Obsługa XML z poziomu języka PHP - metody

```php
<?php
$sxe=simplexml_load_string('<?xml version="1.0"
  encoding="UTF-8"?><studenci></studenci>');
$sxe->addAttribute('Grupa','314A');
$s=$sxe->addChild('student');
$s->addChild('nazwisko','Kowalski');
$s->addChild('imie','Jan');
$p=$s->addChild('przedmiot');
$n=$p->addChild('Nazwa','PAI');
$n=$p->addChild('Nazwa','FIZYKA');
$fp = fopen("plik.xml", "w");
fputs($fp,$sxe->asXML());
?>
```

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
▼<studenci Grupa="314A">
  ▼<student>
     <nazwisko>Kowalski</nazwisko>
     <imie>Jan</imie>
    ▼<przedmiot>
       <Nazwa>PAI</Nazwa>
       <Nazwa>FIZYKA</Nazwa>
     </przedmiot>
   </student>
 </studenci>
```

Obsługa XML – przykład baza pracowników

- Pracownik
  - Id — identyfikator pracownika
  - Imie — Imię pracownika
  - Nazwisko — Nazwisko pracownika
  - Wiek — Liczba lat pracownika
- Stanowisko
  - Id — identyfikator stanowiska
  - Nazwa — Nazwa stanowiska
  - Pensja — Pensja na stanowisk

Obsługa XML – projekt bazy XML

- Pracowanie (który pracownik gdzie pracuje)
  - Id — identyfikator zatrudnienia
  - Pracownik — id pracownika
  - Stanowisko — id stanowiska
  - Lata — ile lat
  - Miesiace — ile miesięcy

Obsługa XML – przykład baza pracowników

- Pracownik
  - Id — identyfikator pracownika (atrybut kluczowy)
  - Imie — Imię pracownika (węzeł)
  - Nazwisko — Nazwisko pracownika (węzeł)
  - Wiek — Liczba lat pracownika (węzeł)
- Stanowisko
  - Id — identyfikator stanowiska (atrybut kluczowy)
  - Nazwa — Nazwa stanowiska (węzeł)
  - Pensja — Pensja na stanowisk (węzeł)

Obsługa XML – przykład baza pracowników

- Pracowanie (który pracownik gdzie pracuje)
  - ✓ Id identyfikator zatrudnienia (atrybut)
  - ✓ Pracownik id pracownika (atrybut klucz z pracownik)
  - ✓ Stanowisko id stanowiska (atrybut klucz ze stanowisko)
  - ✓ Lata ile lat (węzeł)
  - ✓ Miesiace ile miesiący (węzeł)

Obsługa XML – przykład baza pracowników

```
<pracownicy>
 <pracownik id="P1">
  <imie>Adam</imie>
  <nazwisko>Małysz</nazwisko>
  <wiek>36</wiek>
 </pracownik>
 <pracownik id="P2">
  <imie>Ewa</imie>
  <nazwisko>Nowak</nazwisko>
  <wiek>28</wiek>
 </pracownik>
 <pracownik id="P3">
  <imie>Kamil</imie>
  <nazwisko>Kowalski</nazwisko>
  <wiek>23</wiek>
 </pracownik>
 <pracownik id="P4">
  <imie>Maciek</imie>
  <nazwisko>Kowal</nazwisko>
  <wiek>34</wiek>
 </pracownik>
 <pracownik id="P5">
  <imie>Jakub</imie>
  <nazwisko>Michno</nazwisko>
  <wiek>49</wiek>
 </pracownik>
 <pracownik id="P6">
```

```
▼<pracownicy>
  ▼<pracownik id="P1">
      <imie>Adam</imie>
      <nazwisko>Małysz</nazwisko>
      <wiek>36</wiek>
    </pracownik>
  ▼<pracownik id="P2">
      <imie>Ewa</imie>
      <nazwisko>Nowak</nazwisko>
      <wiek>28</wiek>
    </pracownik>
  ▼<pracownik id="P3">
      <imie>Kamil</imie>
      <nazwisk
      <wiek>2
    </pracown
  ▼<pracownik
      <imie>Ma
      <nazwisk
      <wiek>34
    </pracownik>
  ▼<pracownik id="P5">
      <imie>Jakub</imie>
      <nazwisko>Michno</nazwisko>
      <wiek>49</wiek>
```

```
<!ELEMENT pracownicy (pracownik+)>
<!ELEMENT pracownik (imie, nazwisko, wiek)>
<!ATTLIST pracownik id ID #REQUIRED>
<!ELEMENT imie (#PCDATA)>
<!ELEMENT nazwisko (#PCDATA)>
<!ELEMENT wiek (#PCDATA)>
```

Obsługa XML – przykład baza pracowników

```xml
<stanowiska>
 <stanowisko id="S1">
  <nazwa>Kierownik</nazwa>
  <pensja>4000</pensja>
 </stanowisko>
 <stanowisko id="S2">
  <nazwa>Sprzątacz</nazwa>
  <pensja>2500</pensja>
 </stanowisko>
 <stanowisko id="S3">
  <nazwa>Księgowy</nazwa>
  <pensja>3670</pensja>
 </stanowisko>
 <stanowisko id="S4">
  <nazwa>Kasjer</nazwa>
  <pensja>2720</pensja>
 </stanowisko>
 <stanowisko id="S5">
  <nazwa>Magazynier</nazwa>
  <pensja>3100</pensja>
 </stanowisko>
 <stanowisko id="S6">
  <nazwa>Pracownik Sezonowy</nazwa>
  <pensja>2040</pensja>
 </stanowisko>
</stanowiska>
```

```xml
<stanowiska>
 <stanowisko id="S1">
  <nazwa>Kierownik</nazwa>
  <pensja>4000</pensja>
 </stanowisko>
 <stanowisko id="S2">
  <nazwa>Sprzątacz</nazwa>
  <pensja>2500</pensja>
 </stanowisko>
 <stanowisko id="S3">
  <nazwa>Księgowy</nazwa>
  <pensja>3670</pensja>
 </stanowisko>
 <stanowisko id="S4">
  <nazwa>Kasjer</nazwa>
  <pensja>2720</pensja>
 </stanowisko>
</stanowiska>
```

```
<!ELEMENT stanowiska (stanowisko+)>
<!ELEMENT stanowisko (nazwa, pensja)>
<!ATTLIST stanowisko id ID #REQUIRED>
<!ELEMENT nazwa (#PCDATA)>
<!ELEMENT pensja (#PCDATA)>
```

Obsługa XML – przykład baza pracowników

```
<pracowanie>
 <pracuje id="PR1" pracownik="P2" stanowisko="S2">
  <lata>8</lata>
  <miesiace>5</miesiace>
 </pracuje>
 <pracuje id="PR2" pracownik="P5"
  <lata>1</lata>
  <miesiace>3</miesiace>
 </pracuje>
 <pracuje id="PR3" pracownik="P1"
  <lata>12</lata>
  <miesiace>9</miesiace>
 </pracuje>
 <pracuje id="PR4" pracownik="P3"
  <lata>4</lata>
```

```
▼<pracowanie>
  ▼<pracuje id="PR1" pracownik="P2" stanowisko="S2">
     <lata>8</lata>
     <miesiace>5</miesiace>
   </pracuje>
  ▼<pracuje id="PR2" pracownik="P5" stanowisko="S6">
     <lata>1</lata>
     <miesiace>3</miesiace>
   </pracuje>
  ▼<pracuje id="PR3" pracownik="P1" stanowisko="S4">
     <lata>12</lata>
     <miesiace>9</miesiace>
```

```
<!ELEMENT pracowanie (pracuje+)>
<!ELEMENT pracuje (lata, miesiace)>
<!ATTLIST pracuje id ID #REQUIRED>
<!ATTLIST pracuje pracownik IDREF #REQUIRED>
<!ATTLIST pracuje stanowisko IDREF #REQUIRED>
<!ELEMENT lata (#PCDATA)>
<!ELEMENT miesiace (#PCDATA)>
```

Obsługa XML – przykład baza pracowników

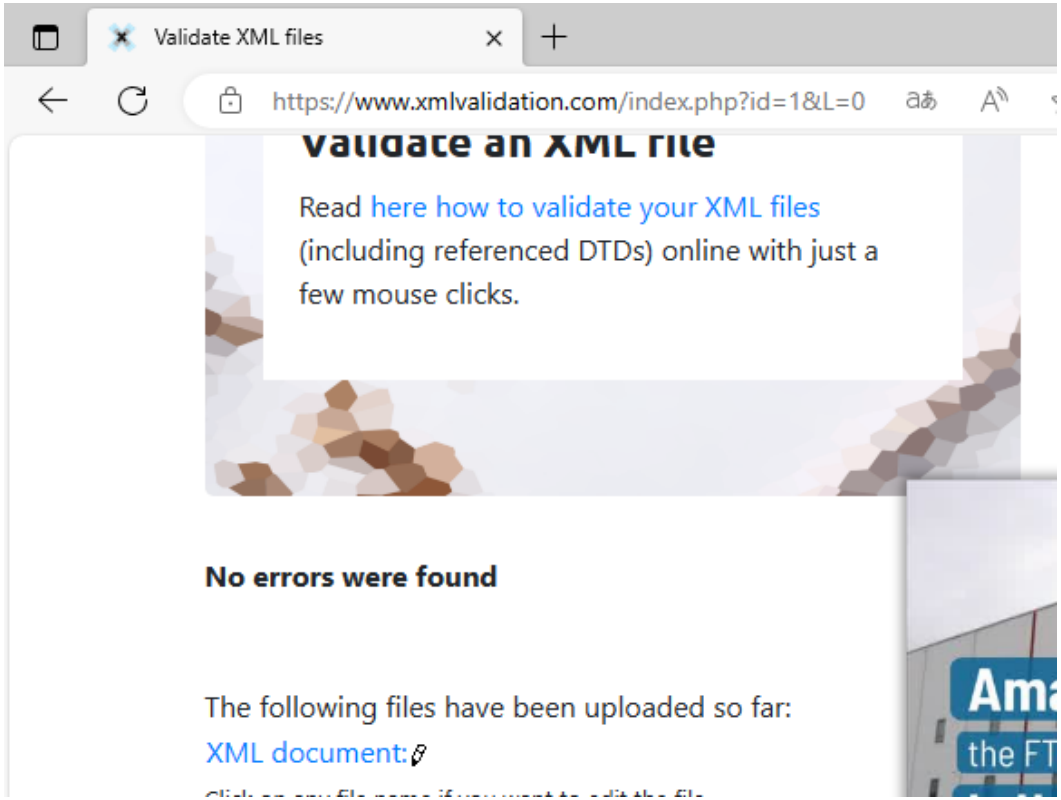
```
▼<baza>
  ▶<stanowiska>
    ...
    </stanowiska>
  ▶<pracownicy>
    ...
    </pracownicy>
  ▶<pracowanie>
    ...
    </pracowanie>
  </baza>
```

```
<!DOCTYPE baza [
<!ELEMENT baza (stanowiska, pracownicy, pracowanie)>
<!ELEMENT stanowiska (stanowisko+)>
<!ELEMENT stanowisko (nazwa, pensja)>
<!ATTLIST stanowisko id ID #REQUIRED>
<!ELEMENT nazwa (#PCDATA)>
<!ELEMENT pensja (#PCDATA)>
<!ELEMENT pracownicy (pracownik+)>
<!ELEMENT pracownik (imie, nazwisko, wiek)>
<!ATTLIST pracownik id ID #REQUIRED>
<!ELEMENT imie (#PCDATA)>
<!ELEMENT nazwisko (#PCDATA)>
<!ELEMENT wiek (#PCDATA)>
<!ELEMENT pracowanie (pracuje+)>
<!ELEMENT pracuje (lata, miesiace)>
<!ATTLIST pracuje id ID #REQUIRED>
<!ATTLIST pracuje pracownik IDREF #REQUIRED>
<!ATTLIST pracuje stanowisko IDREF #REQUIRED>
<!ELEMENT lata (#PCDATA)>
<!ELEMENT miesiace (#PCDATA)>
]>
```

Obsługa XML – przykład baza pracowników

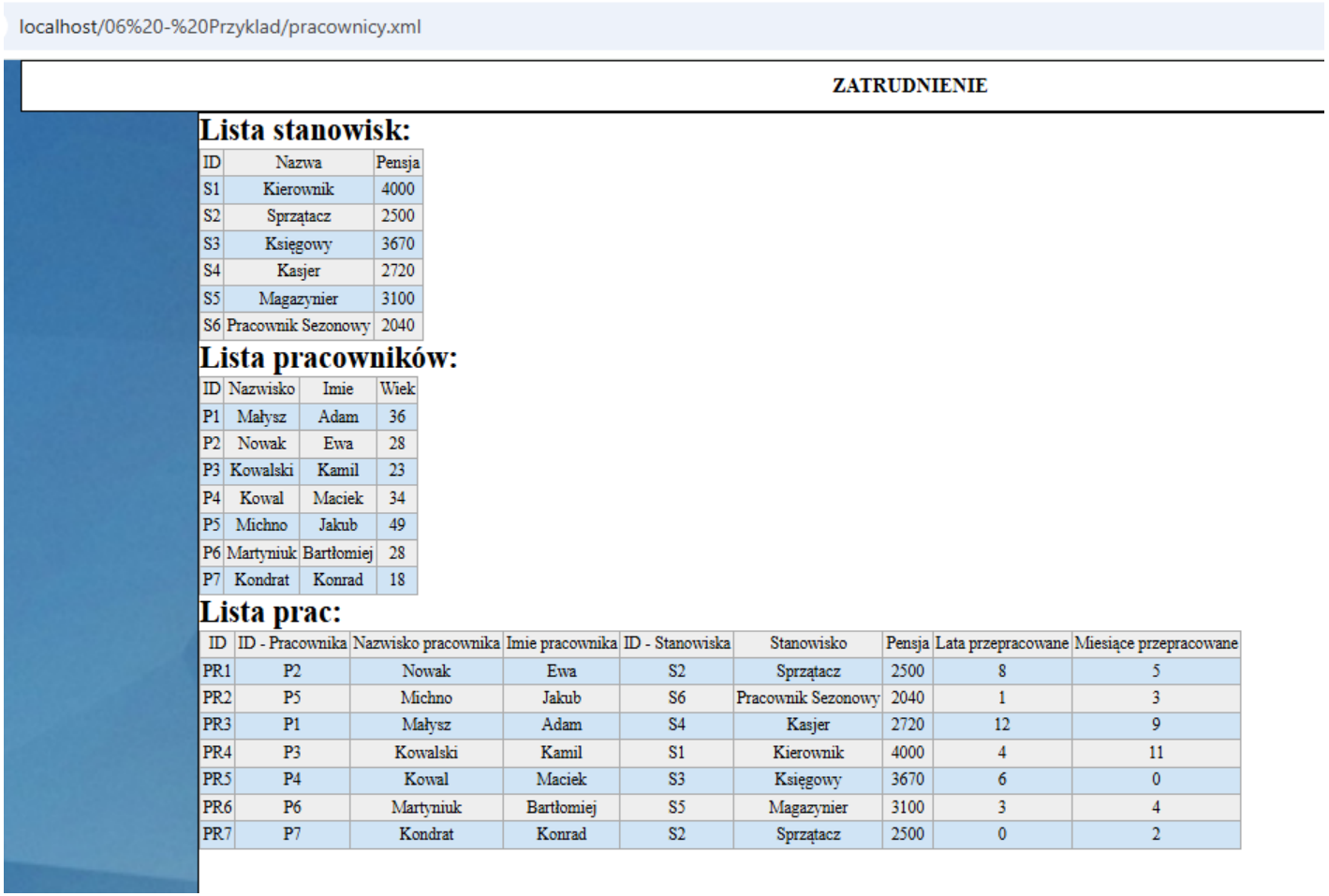
- Sprawdzenie poprawności

  `https://www.xmlvalidation.com/`



Obsługa XML – przykład baza pracowników

- Wyświetlanie XSL



ZATRUDNIENIE

Lista stanowisk:

ID	Nazwa	Pensja
S1	Kierownik	4000
S2	Sprzątacz	2500
S3	Księgowy	3670
S4	Kasjer	2720
S5	Magazynier	3100
S6	Pracownik Sezonowy	2040

Lista pracowników:

ID	Nazwisko	Imie	Wiek
P1	Małysz	Adam	36
P2	Nowak	Ewa	28
P3	Kowalski	Kamil	23
P4	Kowal	Maciek	34
P5	Michno	Jakub	49
P6	Martyniuk	Bartłomiej	28
P7	Kondrat	Konrad	18

Lista prac:

ID	ID - Pracownika	Nazwisko pracownika	Imie pracownika	ID - Stanowiska	Stanowisko	Pensja	Lata przepracowane	Miesiące przepracowane
PR1	P2	Nowak	Ewa	S2	Sprzątacz	2500	8	5
PR2	P5	Michno	Jakub	S6	Pracownik Sezonowy	2040	1	3
PR3	P1	Małysz	Adam	S4	Kasjer	2720	12	9
PR4	P3	Kowalski	Kamil	S1	Kierownik	4000	4	11
PR5	P4	Kowal	Maciek	S3	Księgowy	3670	6	0
PR6	P6	Martyniuk	Bartłomiej	S5	Magazynier	3100	3	4
PR7	P7	Kondrat	Konrad	S2	Sprzątacz	2500	0	2

Obsługa XML – przykład baza pracowników

```
<?xml version="1.0" encoding="UTF-8" ?>
 <xsl:stylesheet version="1.0"
   xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <xsl:template match="/">
    <html>
     <head>
      <meta http-equiv="Content-Type" content="text/html;charset=utf-8" />
      <title>XML PRZYKLADOWA BAZA</title>
      <link rel="stylesheet" type="text/css" href="style.css" />
     </head >
     <body>
      <div id="header">
       <b>ZATRUDNIENIE</b>
      </div>
      <div id="content">
       <xsl:apply-templates/>
      </div>
     </body>
    </html>
 </xsl:template>
```

```
* { margin: 0; padding: 0; box-sizing: border-box; }
body { background-image: url('background.jpg');
       background-repeat: no-repeat;
       background-size: 200%;
       height: 100%;
       width: 100%;
       min-width: 500px;
       max-height: 1000px;}
ul li { text-decoration: none;
        list-style: none;
      }
#header, #topmenu, #content, #footer {
    margin: 0px auto;
    width: 80%;
    height: auto;
    border: 1px solid black;
    background: white;
```

```
<xsl:template match="pracownicy">
 <h2>Lista pracowników:</h2>
 <table>
  <tr>
   <td>ID</td>
   <td>Nazwisko</td>
   <td>Imie</td>
   <td>Wiek</td>
  </tr>
  <xsl:for-each select="pracownik">
   <tr>
    <td><xsl:value-of select="@id"/></td>
    <td><xsl:value-of select="nazwisko"/></td>
    <td><xsl:value-of select="imie"/></td>
    <td><xsl:value-of select="wiek"/></td>
   </tr>
  </xsl:for-each>
 </table>
</xsl:template>
```

Lista pracowników:

ID	Nazwisko	Imie	Wiek
P1	Małysz	Adam	36
P2	Nowak	Ewa	28
P3	Kowalski	Kamil	23
P4	Kowal	Maciek	34
P5	Michno	Jakub	49
P6	Martyniuk	Bartłomiej	28
P7	Kondrat	Konrad	18

Węzeł stanowiska

```
<xsl:template match="stanowiska">
 <h2>Lista stanowisk:</h2>
 <table>
  <tr>
   <td>ID</td>
   <td>Nazwa</td>
   <td>Pensja</td>
  </tr>
  <xsl:for-each select="stanowisko">
   <tr>
    <td><xsl:value-of select="@id"/></td>
    <td><xsl:value-of select="nazwa"/></td>
    <td><xsl:value-of select="pensja"/></td>
   </tr>
  </xsl:for-each>
 </table>
</xsl:template>
```

Lista stanowisk:

ID	Nazwa	Pensja
S1	Kierownik	4000
S2	Sprzątacz	2500
S3	Księgowy	3670
S4	Kasjer	2720
S5	Magazynier	3100
S6	Pracownik Sezonowy	2040

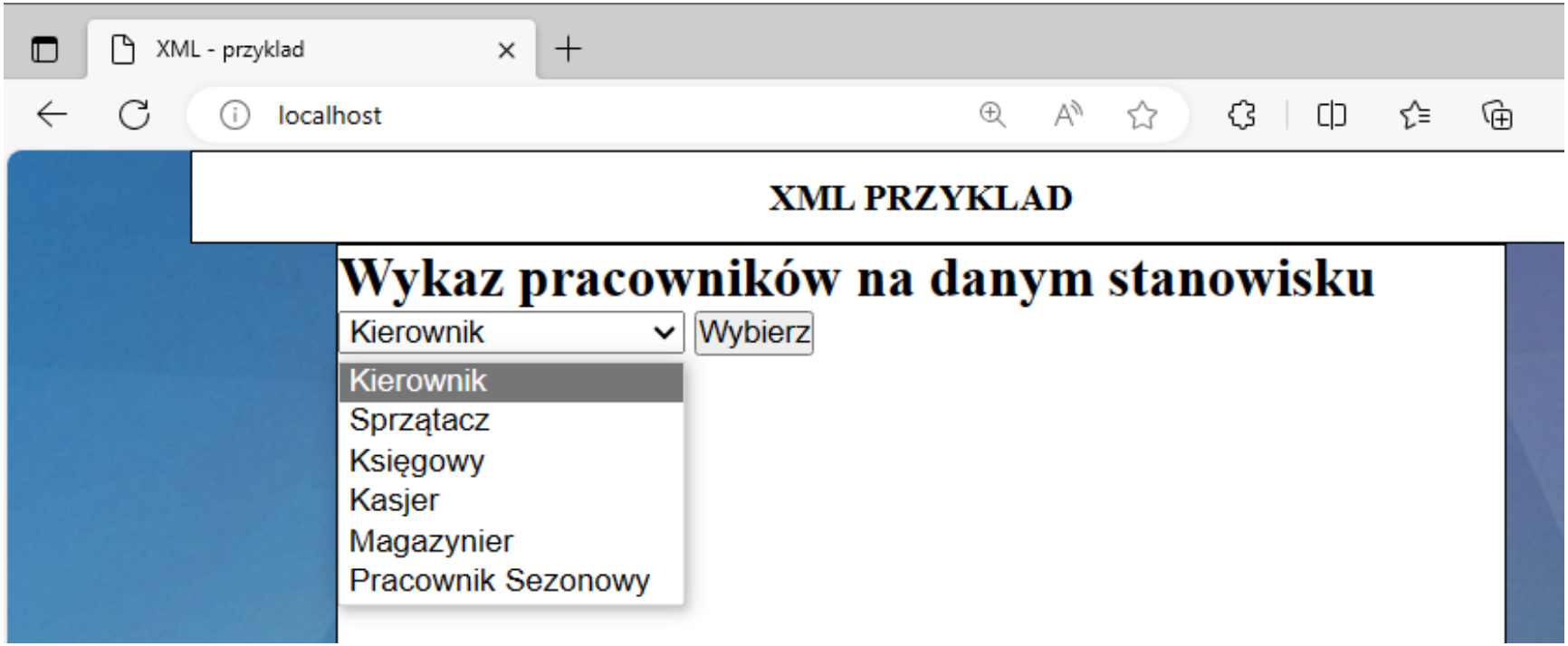
```
<xsl:template match="pracowanie">
<h2>Lista prac:</h2>
 <table>
  <tr>
   <td>ID</td>
   <td>ID - Pracownika</td>
   <td>Nazwisko pracownika</td>
   <td>Imie pracownika</td>
   <td>ID - Stanowiska</td>
   <td>Stanowisko</td>
   <td>Pensja</td>
   <td>Lata przepracowane</td>
   <td>Miesiące przepracowane</td>
  </tr>
  <xsl:for-each select="pracuje">
   <tr>
    <xsl:variable name="id_pracownik">
     <xsl:value-of select="@pracownik"/>
    </xsl:variable>
    <xsl:variable name="id_stanowisko">
     <xsl:value-of select="@stanowisko"/>
    </xsl:variable>
    <td><xsl:value-of select="@id"/></td>
    <td><xsl:value-of select="$id_pracownik"/></td>
    <td><xsl:value-of select="//pracownicy/pracownik[@id=$id_pracownik]/nazwisko"/></td>
    <td><xsl:value-of select="//pracownicy/pracownik[@id=$id_pracownik]/imie"/></td>
    <td><xsl:value-of select="$id_stanowisko"/></td>
    <td><xsl:value-of select="//stanowiska/stanowisko[@id=$id_stanowisko]/nazwa"/></td>
    <td><xsl:value-of select="//stanowiska/stanowisko[@id=$id_stanowisko]/pensja"/></td>
    <td><xsl:value-of select="lata"/></td>
    <td><xsl:value-of select="miesiace"/></td>
   </tr>
  </xsl:for-each>
 </table>
</xsl:template>
```

Lista prac:

ID	ID - Pracownika	Nazwisko pracownika	Imie pracownika	ID - Stanowiska	Stanowisko	Pensja	Lata przepracowane	Miesiące przepracowane
PR1	P2	Nowak	Ewa	S2	Sprzątacz	2500	8	5
PR2	P5	Michno	Jakub	S6	Pracownik Sezonowy	2040	1	3
PR3	P1	Małysz	Adam	S4	Kasjer	2720	12	9
PR4	P3	Kowalski	Kamil	S1	Kierownik	4000	4	11
PR5	P4	Kowal	Maciek	S3	Księgowy	3670	6	0
PR6	P6	Martyniuk	Bartłomiej	S5	Magazynier	3100	3	4
PR7	P7	Kondrat	Konrad	S2	Sprzątacz	2500	0	2

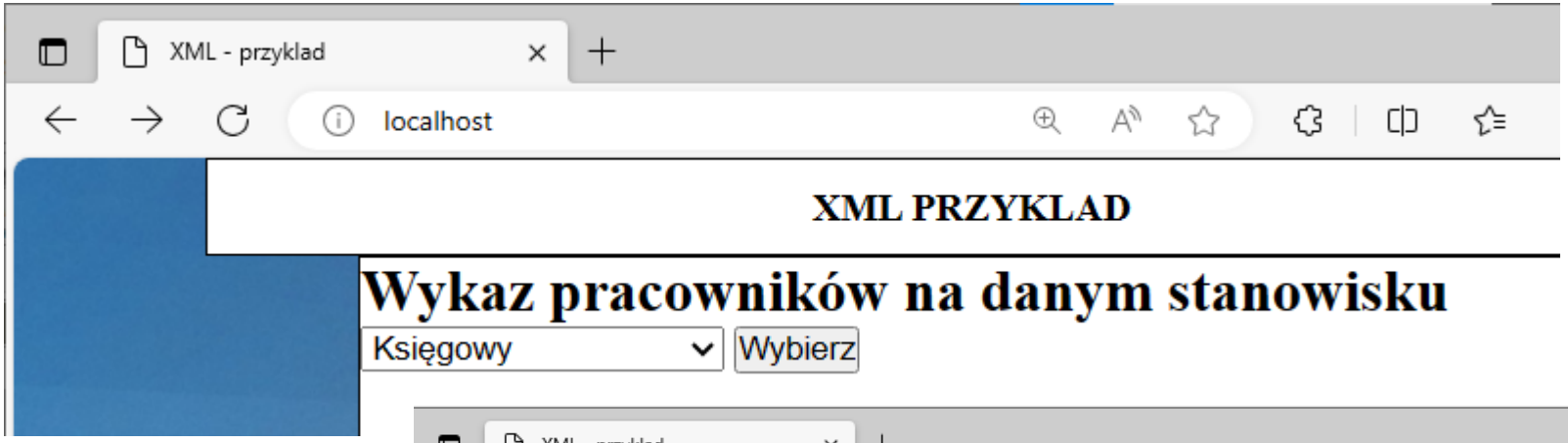
Obsługa XML – przykład baza pracowników

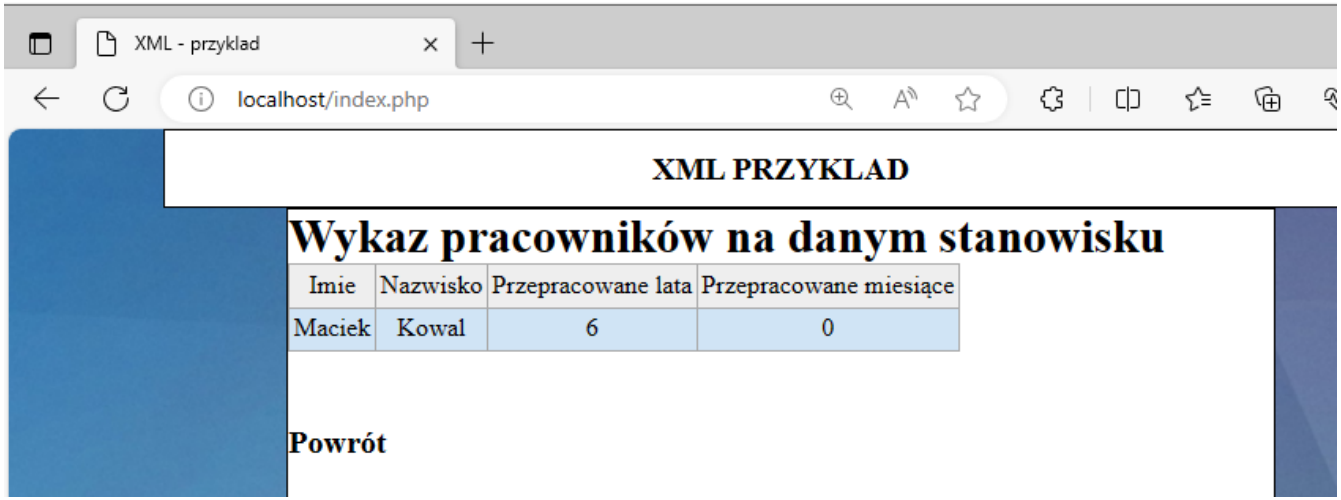
- Wykaz pracowników na danym stanowisku



Obsługa XML – przykład baza pracowników

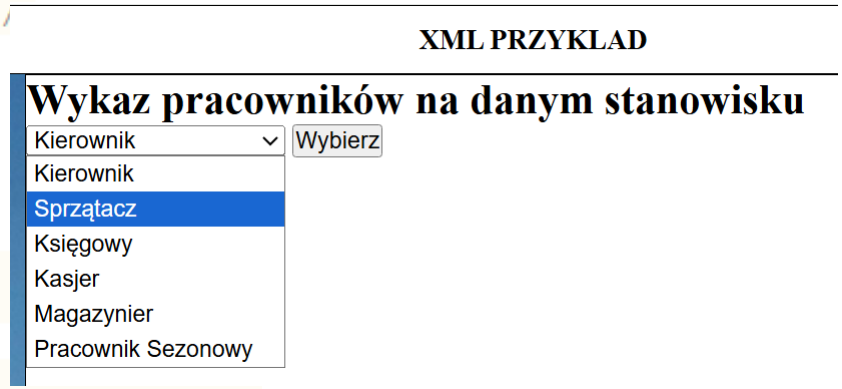
- Wykaz pracowników na danym stanowisku





Obsługa XML – przykład baza pracowników

```php
<?php
echo '<html>
  <head>
   <meta http-equiv="Content-Type" content="text/html;charset=utf-8" >
   <title>XML - przyklad</title>
   <link rel="stylesheet" type="text/css" href="style.css" />
  </head>
 <body>
  <div id="header">
   <b>XML PRZYKLAD</b>
  </div>
  <div id="content">
   <h2>Wykaz pracowników na danym stanowisku</h2>';
   if($_SERVER['REQUEST_METHOD'] == 'GET')
    {echo '<form method="post" action="index.php">
      <select name = "stanowisko">';
       $baza = simplexml_load_file("Pracownicy.xml");
       foreach($baza->stanowiska->stanowisko as $stanowisko)
        {echo '<option value="';
         echo $stanowisko['id'];
         echo '">';
         echo $stanowisko->nazwa;
         echo '</option>';
        };
     echo '</select>
     <input type="submit" value="Wybierz">
    </form>';
  }
```



Obsługa XML – przykład baza pracowników

```
{ echo' <table>
<tr>
<td>Imie</td> <td>Nazwisko</td>
<td>Przepracowane lata</td>
<td>Przepracowane miesiące</td>
</tr>';
$baza = simplexml_load_file('Pracownicy.xml');
foreach($baza->pracowanie->pracuje as $pracuje)
{if($pracuje['stanowisko'] == $_POST['stanowisko'])
{echo '<tr>';
 foreach($baza->pracownicy->pracownik as $pracownik)
 { if((string) $pracownik['id'] == (string) $pracuje['pracownik'])
 { echo "<td>".$pracownik->imie."</td>";
 echo "<td>".$pracownik->nazwisko."</td>";
 }
 }
 echo "<td>".$pracuje->lata."</td>";
 echo "<td>".$pracuje->miesiace."</td>";
 echo "</tr>";
};
};
echo '</table> <br><br>
<h4><a href="index.php">Powrót</a></h4>';
}
echo '</div>
</body>
</html>'
```

Złe rozwiązanie
Sprawdzanie wszystkich

Obsługa XML – przykład baza pracowników

```
{ echo' <table>
<tr>
<td>Imie</td> <td>Nazwisko</td>
<td>Przepracowane lata</td>
<td>Przepracowane miesiące</td>
</tr>';
$baza = simplexml_load_file('Pracownicy.xml');
foreach($baza->pracowanie->pracuje as $pracuje)
{if($pracuje['stanowisko'] == $_POST['stanowisko'])
{echo '<tr>';
 $na= 'pracownicy/pracownik[@id="'.$pracuje['pracownik'].'"]';
 $pracownik=$baza->xpath($na) ;
 echo "<td>".$pracownik[0]->imie."</td>";
 echo "<td>".$pracownik[0]->nazwisko."</td>";
 echo "<td>".$pracuje->lata."</td>";
 echo "<td>".$pracuje->miesiace."</td>";
 echo "</tr>";
};
};
echo '</table> <br><br>
<h4><a href="index1.php">Powrót</a></h4>';
}
echo '</div>
</body>
</html>'
```



XML PRZYKLAD

Wykaz pracowników na danym stanowisku

Imie	Nazwisko	Przepracowane lata	Przepracowane miesiące
Ewa	Nowak	8	5
Konrad	Kondrat	0	2

Powrót